Universidad Internacional de Valencia

Central Vein Sign for Identifying Multiple Sclerosis in Magnetic Resonance Imaging

Titulación:

Master en Inteligencia Artificial
Curso académico 2023 - 2024

Alumno/a: Mendoza Alarcón, Victor Cesar

D.N.I: 183609270

Director/a de TFM: Gema Caballero Muñoz

Convocatoria:

Segunda

11 de noviembre 2024

De:
Planeta Formación y Universidades

The good thing about science
is that it's true whether or not
you believe in it.

Neil deGrasse Tyson

Acknowledgements

First and foremost, I would like to express my deepest gratitude to my parents for their unwavering support throughout my journey in pursuing this master's degree. Their encouragement from the very beginning has been invaluable to me.

I would also like to extend special thanks to my mother for her constant support during the most challenging moments of this project, especially while I was transitioning and moving to Spain. Her belief in me has been a source of strength.

A heartfelt thanks goes to my lovely wife, who encouraged me to pursue my master's studies and provided endless dedication and support throughout this entire process. Her unwavering help and patience have been crucial, and I am deeply grateful for everything she has done for me.

I would also like to express special thanks to my thesis tutor, Gema, for her continuous support of my work despite the challenges we faced. She was always available when I needed guidance or advice, and her encouragement has been instrumental in completing this project.

Finally, I want to extend my deepest thanks to my brother and all my family for their constant support and encouragement throughout this entire process. Their belief in me has meant the world, and I couldn't have done this without them.

Special thanks to my friend Marines. Despite the distance, she always shows me her support, and her words constantly remind me why I chose to start this project and why I need to stay calm to reach my goal

Contents

List of Figures

List of Tables

Abstract

Multiple sclerosis (MS) is a chronic autoimmune disease that affects the central nervous system, leading to a wide range of symptoms unique to each patient, hence it is often referred to as "the disease of a thousand faces." This variability makes accurate diagnosis challenging, as MS can be easily mistaken for other conditions. A promising biomarker for distinguishing MS is the Central Vein Sign (CVS). This thesis explores the integration of deep learning techniques with MRI to detect CVS lesions, aiming to improve the accuracy of MS diagnosis. To undertake this project, it was necessary to develop a custom dataset through careful investigation and the application of image pre-processing techniques to handle diverse images collected from public datasets. We proposed three models with two differnt approaches: a basic 3D-CNN, a model inspired by CVSNet, and a transfer learning approach using ResNet50 with a Lambda layer with and without hyperparameter tuning. Finally, we present the results obtained from our small dataset using these three models and their variants and discuss the challenges encountered throughout the project, setting the stage for future research in this area.

Keywords: Multiple Sclerosis, Central Vein Sign, Magnetic Resonance Imaging, Deep Learning, Convolutional Neural Networks, Medical Imaging, Artificial Intelligence

Introduction

1

Throughout history, humans have been afflicted by a variety of diseases, disorders, and injuries affecting different parts of the body. These conditions can arise from genetic abnormalities, viral infections, or other causes, leading to a range of health complications—sometimes even resulting in death. While medical advancements have enabled us to identify the causes of many ailments, certain conditions, particularly those affecting the brain, remain challenging to diagnose and understand.

Neurology, the branch of medicine that addresses disorders of the nervous system, covers a broad spectrum of conditions impacting the brain, spinal cord, and peripheral nerves. Among these are complex neurological diseases such as Alzheimer's, Parkinson's, and Multiple Sclerosis (MS). These conditions not only pose significant challenges to individual patients but also place a substantial burden on healthcare systems worldwide. Understanding and accurately diagnosing such diseases is crucial for effective management and mitigation of their effects.

Multiple Sclerosis (MS), also known as "the disease of a thousand faces", is a chronic autoimmune disease that typically affects the central nervous system (CNS) and presents a wide range of neurological symptoms that can vary from patient to patient. Correctly diagnosing MS is challenging due to its symptoms overlapping with those of other neurological diseases, such as Neuromyelitis Optica Spectrum Disorder (NMOSD) and small vessel ischemic disease. Current diagnostic methods, including clinical evaluation, cerebrospinal fluid analysis, and conventional MRI, often struggle to differentiate MS from these mimics accurately. Therefore, developing better diagnostic tools and biomarkers is essential for more accurate identification of MS lesions.

Advances in medical imaging and artificial intelligence (AI) are opening new possibilities for improving MS diagnosis. Magnetic Resonance Imaging (MRI) has become a cornerstone in diagnosing and managing MS because it provides detailed images of brain and spinal cord lesions. However, the inclusion of advanced imaging biomarkers like the Central Vein Sign (CVS) and their integration with AI and deep learning (DL) techniques have shown significant potential in enhancing the accuracy and efficiency of MS diagnosis. In particular, DL models can automatically detect these biomarkers, reducing manual intervention, minimizing diagnostic errors, and reducing human error.

This thesis explores the integration of DL models with MRI techniques to improve the detection of CVS for MS diagnosis. This research aims to address the current gaps in MS diagnosis by leveraging state-of-the-art AI techniques to enhance clinical decision-making.

Aims and Objectives

2

The primary aim of this project is to achieve high accuracy in detecting Multiple Sclerosis (MS) using the Central Vein Sign (CVS) from MRI datasets gathered from publicly available sources, as this project is not affiliated with any specific institution. The research is structured around the following six objectives:

1. **Investigate what Multiple Sclerosis (MS) is and how it is diagnosed.**
2. **Examine the role of Magnetic Resonance Imaging (MRI) in detecting MS.**
3. **Determine the importance of the Central Vein Sign (CVS) as a biomarker for diagnosing MS.**
4. **Review previous studies that use CVS with artificial intelligence (AI) to diagnose MS.**
5. **Collect publicly available MRI datasets and identify those containing CVS lesions.**
6. **Develop and implement a deep learning (DL) model to detect CVS Lesion in MS from MRI data.**

Theoretical framework

3

3.1 Multiple Sclerosis

Multiple Sclerosis (MS) is a chronic autoimmune disease characterized by the demyelination of nerve fibers in the central nervous system (CNS). This demyelination disrupts the communication between the brain and the rest of the body, leading to a wide range of neurological symptoms. MS manifests in different forms, which are broadly classified as: relapsing-remitting, secondary progressive, and primary progressive (Ortiz et al., 2016).

In addition to these classifications, there is another category known as Clinically Isolated Syndrome (CIS). CIS refers to a single episode of neurological symptoms suggestive of an inflammatory demyelinating disorder, which may or may not progress to multiple sclerosis. Studies have shown that CIS can be an early indicator of MS, particularly if MRI findings are suggestive of the disease (David H Miller, 2012).

The pathophysiology of MS involves an immune-mediated process in which the body's immune system mistakenly attacks the myelin sheath, a protective layer surrounding nerve fibers. This results in scar tissue (sclerosis) forming along the damaged nerves, leading to the symptoms of MS, which can include fatigue, motor and sensory disturbances, and cognitive impairment. The impact of MS on individuals varies greatly, with some experiencing mild symptoms and others facing significant disability.

3.2 Diagnosing Multiple Sclerosis

The process of accurately diagnosing Multiple Sclerosis (MS) is complex due to the variability of symptoms, which can present differently in each individual. Given the overlapping features of MS with other demyelinating diseases, it is crucial to approach the diagnosis with caution to avoid misdiagnosis. To ensure an accurate diagnosis, it is essential to distinguish MS from other conditions through a comprehensive diagnostic process, which includes both the identification of MS and the differentiation from similar diseases. This is where the concepts of diagnosis and differential diagnosis become particularly important.

Since the 1950s, techniques for diagnosing MS have evolved to increase their sensitivity and specificity. As a result, several diagnostic criteria have been developed, including those proposed by Schumacher, Poser, and McDonald (Ömerhoca et al., 2018), with the latter undergoing multiple revisions over time.

The evolution of these diagnostic methods has not only enabled the detection of potential MS cases but also the confirmation of whether a hypothetical case is indeed MS. Primary diagnostic tools include blood tests (e.g., hemogram, renal and liver function tests, lipid panels), MRI of the brain, cervical, and thoracic spine, lumbar puncture for cerebrospinal fluid (CSF) analysis (one of the most critical tests), and optical coherence tomography (OCT), particularly in patients with optic neuritis.

In addition to these primary tests, secondary tests can provide further diagnostic insight. These include evoked potentials (EP), OCT (which may be repeated if the patient initially lacks optic neuritis), urodynamic testing, and cognitive testing. These secondary tests can be highly valuable in refining the MS diagnosis.

Given the potential for other demyelinating diseases, such as Acute Disseminated Encephalomyelitis or Idiopathic Inflammatory Non-Demyelinating Diseases, it is essential to employ differential diagnosis methods to avoid misdiagnosis. This may involve additional biochemical tests, infectious disease screenings, angiography, biopsies, chest X-rays, and other evaluations. The results of these tests can help to accurately diagnose the specific type of MS or related disease, allowing for appropriate treatment.

For a more in-depth discussion of these diagnostic techniques, refer to (Ömerhoca et al., 2018).

3.3 Magnetic Resonance Imaging

MRI is a non-invasive imaging technology that produces detailed three-dimensional images of internal anatomical structures. It is widely used to detect, diagnose, and monitor various diseases, particularly those affecting soft tissues like the brain and spinal cord. *MRIs employ powerful magnets which produce a strong magnetic field (also know as Tesla or T) that forces protons in the body to align with that field* (National Institute of Biomedical Imaging and Bioengineering (NIBIB), 2023). When the magnetic field is turned off, the protons release energy as they return to their original alignment, and MRI sensors detect this energy to create detailed images.

During an MRI scan, the patient is placed inside a large magnet and must remain still to avoid blurring the images. In some cases, a contrast agent is administered intravenously before the scan. This agent helps enhance the visibility of certain tissues or abnormalities by altering the magnetic properties of water molecules in the body, leading to clearer images.

MRIs are particularly effective at distinguishing between white and gray matter in the brain, which is helpful for diagnosing conditions like tumors and aneurysms. However, because MRIs use powerful magnets, they pose risks for individuals with metallic implants, those who are pregnant, have claustrophobia, are sensitive to loud noises, or have renal issues that make them sensitive to contrast agents.

3.3.1 MRI Lesions in MS

MRIs are powerful tools for detecting, diagnosing, and monitoring various diseases. As discussed in Section 3.3, this technology can be used to identify different conditions. In our case, the focus will be on MRI images related to MS. The appearance of these images can vary depending on the part of the body being examined and the strength of the magnetic field used, as this can affect the resulting imaging.

Since the revision of the McDonald criteria in 2017, there are general guidelines on how MRI can be used to detect MS, which are detailed in (Filippi et al., 2019). Based on these guidelines, we can categorize different types of lesions and determine if they are applicable for the formal diagnosis of MS.

Lesions in MS tend to affect specific regions of the white matter, such as the periventricular and juxtacortical areas, the corpus callosum, infratentorial regions, and the spinal cord. These lesions are not restricted to one hemisphere, but their distribution is usually mildly asymmetric in the early stages. For a lesion to be considered a criterion for diagnosing MS, it must be at least $3mm$ in its long axis and is generally round to ovoid in shape. As defined in (Filippi et al., 2019), a lesion is *an area of focal hyperintensity on a T2-weighted (T2, T2-FLAIR, or similar) or a proton density (PD)-weighted sequence.*

As mentioned earlier, some lesions are part of the diagnostic criteria, while others are not or are areas of ongoing research. In our study, we will focus solely on central vein sign lesions in Section 3.4. For a more in-depth discussion of each lesion type, refer to (Filippi et al., 2019). The following list categorizes the lesions:

Part of the diagnostic criteria:

- Periventricular Lesions
- Juxtacortical or cortical lesions
- Infratentorial lesions
- Spinal cord lesions
- Gadolinium-enhancing lesions

Not part of the diagnostic criteria:

- Optic Nerve

Ongoing research:

- Central Vein Sign
- Subpial Demyelination
- Smoldering/slowly expanding lesions

3.4 Central Vein Sign

CVSis an MRI biomarker proposed to help distinguish MS from other diseases that mimic its presentation, such as NMOSD, which is described as *a CNS autoimmune disease that predominantly affects the optic nerves and spinal cord* (Sati et al., 2016). In their early stages, these conditions can be challenging to differentiate.

The identification of CVS is important because it increases the specificity of diagnosing MS. This is due to the fact that *lesions have been described histopathologically as occurring around central veins* (Ontaneda et al., 2021). These lesions are more commonly found in the periventricular area and decrease as one moves closer to the neocortex (including deep white matter lesions, juxtacortical lesions, mixed grey and white matter lesions, and intracortical lesions).

The extensive literature on central vein imaging in MS suggests that CVS can detect around 85% of the lesions in the white matter of MS patients, compared to other diseases, such as migraines (34%), small vessel ischemic disease (8%), and other inflammatory or autoimmune diseases (14%). This makes CVS an excellent addition to the diagnostic criteria, as it increases both the sensitivity and specificity of MS detection.

The Central Vein Sign is best visualized using specialized MRI sequences that are sensitive to both white matter lesions and the veins within them, such as:

T2-Weighted Imaging (T2-WI): This MRI sequence is highly sensitive to magnetic susceptibility differences caused by the presence of blood vessels. It helps visualize veins as linear dark structures within hyperintense lesions.

Susceptibility-Weighted Imaging (SWI): This advanced MRI technique enhances the contrast between veins and surrounding tissues, making it one of the most effective methods for detecting the central vein sign.

T2-Fluid Attenuated Inversion Recovery (T2-FLAIR): When combined with T2* or SWI, T2-FLAIR can help visualize both the lesions and the central veins running through them, as the sequence suppresses the free water signal, such as cerebrospinal fluid, to provide a clearer view of white matter lesions.

FLAIR (FLAIR-Star): This is a combined sequence that merges T2-FLAIR and T2* or SWI images, enhancing both lesion and vein visualization in a single image, providing a more comprehensive view.

T1-Weighted Imaging (T1-WI): Although not commonly used to directly visualize the Central Vein Sign (CVS), T1-weighted imaging provides excellent anatomical detail and is valuable for distinguishing lesion borders and identifying brain atrophy. It can be used in combination with other sequences to assess the relationship between lesions and veins.

3.5 Artificial Intelligence in Medical Imaging

The use of Artificial Intelligence in medical imaging has become increasingly valuable for diagnosing complex diseases, such as multiple sclerosis. Advanced techniques, like Deep Learning (DL), enable the analysis and processing of large volumes of medical images with high accuracy, provided the data is properly managed. For this reason, it is essential to understand Deep Learning, their role in medical diagnosis, and specifically how they are applied in MRI to distinguish various neurological diseases.

Deep learning is a subfield of machine learning (ML) that, unlike traditional ML, usually requires more data and primarily uses Neural Networks (NN) as its core. The key difference between DL and traditional ML is seen in fields such as image processing, signal processing, and others where the amount of data is large. In image processing, traditional ML typically requires handcrafted feature extraction and selection, which can be time-consuming and prone to human error.

While traditional ML can be useful in certain fields or projects where errors are less critical, this is not the case in the medical field. Medical images, such as tomography, MRI, and X-rays, correspond to crucial features of the human body. Though there are advanced ML methods available today, the rapid development of DL techniques has led to increased accuracy and efficiency in analyzing medical images. This improvement enhances the quality of care and saves time because DL models can automatically extract and select relevant features from images without manual intervention.

3.6 Machine Learning and Deep Learning in Medical Images

As discussed in Section 3.5, there are different techniques in ML that show good results for medical images. For example, *Support Vector Machines (SVMs), decision trees, random forests, and logistic regression, have shown success in tasks such as image segmentation, object detection, and disease classification* (Zhang y Qie, 2023). There are also DL models designed for similar purposes, often yielding better results.

In the case of DL, particularly with images, a significant amount of data is required to achieve good results. However, obtaining a large number of specific medical images is often challenging. Although techniques such as image augmentation are employed to artificially increase dataset size, caution is necessary. For instance, simple augmentation techniques like flipping the image can lead to incorrect diagnoses. In such situations, increasing the complexity of neural networks or using more specialized neural networks for the specific task becomes essential.

3.6.1 Deep Learning Architectures for Medical Imaging

Some of the DL techniques frequently used in medical imaging include Convolutional Neural Networks (CNNs). CNNs are widely used for image analysis because they were specifically created for this purpose, making them a good fit for handling various types of images. CNNs are composed of layers, which typically include convolutional, pooling, and fully connected layers:

Convolutional Layer: Extracts different features in an image, such as edges, corners, and other useful patterns.

Pooling Layer: Reduces the spatial dimensions of the image, helping to reduce overfitting and improve computational efficiency.

Fully Connected Layer: Combines local features into global patterns, enabling the network to perform tasks such as classification and other analyses.

Other neural networks that can be used include Recurrent Neural Networks (RNNs), which are designed to handle sequential data and learn patterns in time-dependent data. RNNs are particularly suitable for tasks involving time-series or sequential information. Another example is Generative Adversarial Networks (GANs), which are a type of network capable of generating realistic samples for complex data. GANs consist of a minimax game between two models. For more in-depth coverage of the limitations and challenges of these methods, refer to Section 3.5.

3.7 State Of The Art

3.7.1 Automated Integration of Multimodal MRI for the Probabilistic Detection of the Central Vein Sign in White Matter Lesions

In (Dworkin et al., 2018), the authors present an automated method to detect the CVS using multimodal MRI. The proposed method calculates the probability of the CVS biomarker in each lesion. The study demonstrates that patients with MS had significantly higher values of this biomarker compared to those without MS, achieving strong discriminative ability with an area under the curve (AUC) of 0.88. This indicates the method's potential for clinical application of CVS as a diagnostic biomarker for MS. For our project, this paper highlights the potential of CVS as a discriminative marker for diagnosing MS.

3.7.2 Evaluation of the Central Vein Sign as a Diagnostic Imaging Biomarker in Multiple Sclerosis

The study conducted by Sinnecker et al. in (Sinnecker et al., 2019) was a multicenter study aimed at evaluating the diagnostic accuracy of various CVS criteria using 3T MRI. The results showed that CVS has a high specificity of 82.9% but a moderate sensitivity of 68.1% when distinguishing MS from non-MS diagnoses. This study suggests that multicenter trials are necessary to validate CVS as a reliable biomarker for differentiating MS from other conditions in clinical practice. This relates to our findings that, while certain techniques have high specificity, challenges remain with sensitivity when distinguishing MS from other diseases.

3.7.3 CVSnet: A Machine Learning Approach for Automated Central Vein Sign Assessment in Multiple Sclerosis

CVSnet, developed by Maggi et al. in (Maggi et al., 2020), is a 3D-CNN designed to automatically detect the CVS in white matter MS lesions. This CNN was trained on MRI data from different imaging centers and achieved a high lesion-wise balanced accuracy of 81% and a subject-wise balanced accuracy of 91% on the test set and 81% on the validation set. CVSnet demonstrated its ability to reduce the time required for CVS assessment compared to manual evaluation and outperformed a state-of-the-art vesselness filtering method. The study also highlights CVSnet's potential for future work (and this project) in more efficient and accurate differentiation of MS from its mimics.

3.7.4 Central Vein Sign Profile of Newly Developing Lesions in Multiple Sclerosis

In (Al-Louzi et al., 2022), the authors conducted a longitudinal cohort study involving patients with newly detected T2 or enhancing lesions based on radiology reports and/or longitudinal subtraction imaging, where each lesion was evaluated for the Central Vein Sign (CVS). Their results show that, over a median period of three years, most newly appearing T2 or enhancing lesions were CVS-positive (68%), and only 48% of patients developed new CVS-positive lesions. This finding suggests that CVS remains consistent as lesions progress, enabling the tracking of multiple sclerosis progression and serving as a valuable biomarker for disease evolution over time. From this, we can see CVS's potential not only to detect MS from a single T2-weighted image but also to track progression over time, making CVS valuable for ongoing monitoring.

3.7.5 Cortical lesions, central vein sign, and paramagnetic rim lesions in multiple sclerosis: Emerging machine learning techniques and future avenues

In their review, La Rosa et al. (La Rosa et al., 2022) discuss emerging machine learning techniques, including deep learning, in combination with biomarkers for multiple sclerosis. The team focused on three specific biomarkers: Cortical Lesions, the Central Vein Sign, and Paramagnetic Rim Lesions, exploring the techniques applied to each and the outcomes of these studies.

They found that these approaches have the potential to provide standardized identification of biomarkers. However, several limitations must be addressed first, such as establishing standardized protocols and criteria across biomarkers, improving validation with multicenter datasets that use different protocols, and extending frameworks to include longitudinal data. This evidence strengthens the case that using CVS to detect MS is a promising approach that warrants further research for potential inclusion in the McDonald criteria for diagnosis.

Methodology

4

4.1 Programming Languages and Tools

In this thesis, we primarily use the Python programming language due to its extensive support for scientific computing and deep learning libraries. Specifically, we utilize libraries such as TensorFlow, Keras, Keras Tuner and Scikit-learn, along with more general-purpose libraries like NumPy and Pandas, if necessary.

Since this research is conducted independently, without institutional funding, I do not have access to high-performance computing resources. The experiments were conducted on my personal laptop, which has 16 GB of RAM, an Nvidia GPU with 4 GB of VRAM, and an AMD Quad-core R5-2500U processor. As a result, the outcomes of this study are somewhat constrained by the limited computational resources available.

All the notebooks used in this research, along with instructions for replication and the necessary dependencies, will be made available at https://github.com/Kinozuko/viu_master_thesis.

4.2 MRI Data Collection

As mentioned in previous sections, this project is conducted independently and is not affiliated with any institution. Therefore, the resources for this study are limited to publicly available, open-source datasets found online. After searching for suitable MRI datasets, we identified four different datasets that could potentially be useful for this project. These datasets will need to be processed and evaluated to determine their relevance to the goals of this thesis. The details of each dataset are explained in the following sections.

4.2.1 MS Brain MRI with Lesion Segmentation

The first dataset we found is hosted on Mendeley, a reference management tool and academic network commonly used by researchers to organize, share, and cite research papers and datasets, as well as collaborate with others.

The dataset is titled **Brain MRI Dataset of Multiple Sclerosis with Consensus Manual Lesion Segmentation and Patient Meta Information** (M. Muslim, 2022). It consists of multi-sequence MRI data from 60 patients with MS and includes various metadata for each patient,

although we will not be using the metadata in this project. Instead, our focus will be on the MRI images themselves. The lesion segmentation was performed by expert radiologists and neurologists across three MRI sequences: T1-weighted, T2-weighted, and FLAIR.

This dataset was created to support a range of research endeavors, such as automated MS-lesion segmentation, prediction, or any other relevant analyses. The dataset can be accessed at https://data.mendeley.com/datasets/8bctsm8jz7/1.

4.2.2 Tremor Investigation in MS Using MRI

The dataset titled **Investigation of Tremor in Multiple Sclerosis (MS) via Magnetic Resonance Imaging (MRI)** (Egan, 2013) is owned by Professor Gary Egan of The University of Melbourne and is hosted on Research Data Australia.

This dataset focuses on investigating tremor in MS using techniques such as diffusion and conventional contrast imaging. It contains images from 14 subjects across 12 studies, totaling 126 datasets with a combined size of approximately 1.52 GB.

Although this dataset appears to be a promising fit for the project due to its variety of images, access is restricted and requires negotiation with Professor Gary Egan to discuss terms, conditions, and potential costs. The dataset can be accessed at https://researchdata.edu.au/investigation-tremor-multiple-imaging-mri/185799.

4.2.3 MS Lesion Segmentation in MRI

The dataset titled MRI Lesion Segmentation in Multiple Sclerosis Database (Loizou et al., 2013a, 2011, 2013b, ress; Loizou, 2018), authored by Professor Christos P. Loizou, is hosted on Academic Torrents.

The dataset includes a folder called "IMT-Segmentation," which contains 38 subfolders corresponding to 38 patients. For each patient, there is an "MRI TIFF" folder that contains images from both the first and second examinations (0 months and 6/12 months), along with lesion segmentations in .plq format, which can be used for analysis in MATLAB.

This dataset may be useful for the project as it contains information about the progression of patients and may include early-stage CVS lesions. The dataset can be accessed at https://academictorrents.com/details/e08155e5022d688fea00319bd2ead4f0f703f5bb.

4.2.4 Shifts Multiple Sclerosis Lesion Segmentation

This dataset is based on the previous work titled **Shifts 2.0: Extending The Dataset of Real Distributional Shifts** (Malinin et al., 2022), which focused on improving model robustness in handling distributional shifts and managing uncertainty. The dataset is licensed under CC BY NC SA 4.0. This work (Malinin et al., 2022) *is supported by the Hasler Foundation, Cambridge University Press, Cambridge Assessment, and DeepSea.*

The dataset is divided into two parts. While both parts are available online, access to the first part is restricted and requires a request based on certain conditions. This part can be found

at https://zenodo.org/records/7051658. On the other hand, the second part is openly accessible and can be found at https://zenodo.org/records/7051692.

4.3 Pre-processing MRI Data

As mentioned in section 4.2, we identified four different datasets (or five, if we count the one that is divided into two parts). For this study, we will focus only on the datasets mentioned in sections 4.2.1, 4.2.3, and the second part of section 4.2.4, as these are the datasets with open access and do not require prior access requests.

Since it was not possible to find MRI datasets specifically containing lesions near the (CVS) for MS, we manually selected the images from the collected datasets that are relevant to this project.

4.3.1 Dataset Download and Initial Investigation

After downloading the datasets from sections 4.2.1, 4.2.3, and the second part of 4.2.4, we created a folder named "datasets" to store the data. We also created a script called **analyse_datasets.py** (see Code A.1 for more info) to summarize the file formats available across the folders and their counts, allowing us to identify which files we need to keep or investigate further.

Across all the folders, we found 10 different file formats, with a total of 4625 elements. The count of files per extension is listed in table 4.1.

Extension	Count
.md	1
.xls	1
.xlsx	2
.txt	3
.DS_Store	7
.nii	360
.gz	636
.bmp	811
.TIF	1027
.plq	1777

Table 4.1: File Extensions and Total Counts

As we can see, the datasets contain various file formats, but not all of them are relevant. Therefore, we need to select only the important ones. After searching online for information about these file extensions, we identified that **.nii** and **.TIF** are commonly used in medical imaging and medical research. Additionally, the **.gz** format is a compressed file, and it may be worth investigating whether it contains any useful images.

On the other hand, common file formats like **.xlsx** and **.xls** might be worth checking to see if they contain any relevant information. The **.plq** format is a MATLAB format used to store

images of lesions, as mentioned in section 4.2.3.

To further investigate the file extensions and their locations within the datasets, we extended the script **analyse_datasets.py** to display the file extensions based on the root folder of each dataset, as shown in table 4.2. This allowed us to prioritize which folders to check first, based on the number of elements they contain.

Root Folder	Extension	Count	Subfolders
datasets/Brain MRI Dataset of Multiple Sclerosis	.xlsx .nii	2 360	60
datasets/shifts_ms_pt2	.txt .md .DS_Store .gz	2 1 7 636	2
datasets/MRIFreeDataset	.xls .txt .bmp .TIF .plq	1 1 811 1027 1777	1

Table 4.2: Root Folders, File Extensions, Counts, and Sub-folders

From now on, we will refer to the datasets as follows: **dataset_1** for the **Brain MRI Dataset of Multiple Sclerosis**, **dataset_2** for **shifts_ms_pt2**, and **dataset_3** for the **MRIFreeDataset**. This naming convention will be used throughout this document to avoid repeatedly writing the full names of the downloaded datasets.

4.3.2 Investigating Datasets

4.3.2.1 Dataset 1

For this dataset, as shown in Table 4.2, we identified two **.xlsx** files and several **.nii** files spread across 60 subfolders. The first step was to locate the **.xlsx** files, and we found that they were named as follows:

- Supplementary Table 1 for patient info .xlsx
- Supplementary Table 2 for sequence parameters .xlsx

After examining the files, we found that each file contains different information. The Supplementary Table 1 is related to patient data, including general and clinical information, and it tracks various neurological symptoms. We can infer that this table serves as a comprehensive record of each patient's clinical and neurological status. The columns of this table are shown in Figure 4.1.

On the other hand, Supplementary Table 2 is more focused on the MRI sequence parameters. Based on the column names, we can infer that it contains multiple scan sequences for each patient. The columns of this table are shown in Figure 4.2.

```
Columns for Supplementary Table 1 for patient info .xlsx:

Index(['ID', 'Gender', 'Age', 'Age of onset', 'EDSS',
       'Does the time difference between MRI acquisition and EDSS < two months',
       'Types of Medicines', 'Presenting Symptom',
       ' Dose the patient has Co-moroidity', 'Pyramidal', 'Cerebella',
       'Brain stem', 'Sensory', 'Sphincters', 'Visual', 'Mental', 'Speech',
       'Motor System', 'Sensory System', 'Coordination', 'Gait',
       'Bowel and bladder function', 'Mobility', 'Mental State', 'Optic discs',
       'Fields', 'Nystagmus', 'Ocular Movement', 'Swallowing'],
      dtype='object')
```

Figure 4.1: Columns for Supplementary Table 1 for patient info .xlsx

```
Columns for Supplementary Table 2 for  sequence parameters .xlsx:

Index(['ID', 'Repetition Time  (TR)', 'Echo Time (TE)', 'Slice Thickness ',
       'Spacing Between Slices', 'Repetition Time  (TR).1', 'Echo Time (TE)
       'Slice Thickness .1', 'Spacing Between Slices.1',
       'Repetition Time  (TR).2', 'Echo Time (TE).2', 'Slice Thickness .2',
       'Spacing Between Slices.2'],
      dtype='object')
```

Figure 4.2: Columns for Supplementary Table 2 for sequence parameters .xlsx

Although this information appears interesting, we will disregard it for now, as we are focusing solely on the MRI data. Upon examining the dataset in more detail, we discovered sub-folders named in the format Patient-x, where x indicates the patient ID. Each folder contains four ***.nii** files, which are exactly what we need for this project. Additionally, each of these files has a different name corresponding to various scans, such as FLAIR. For example, the files for Patient-1 are shown in Figure 4.3.

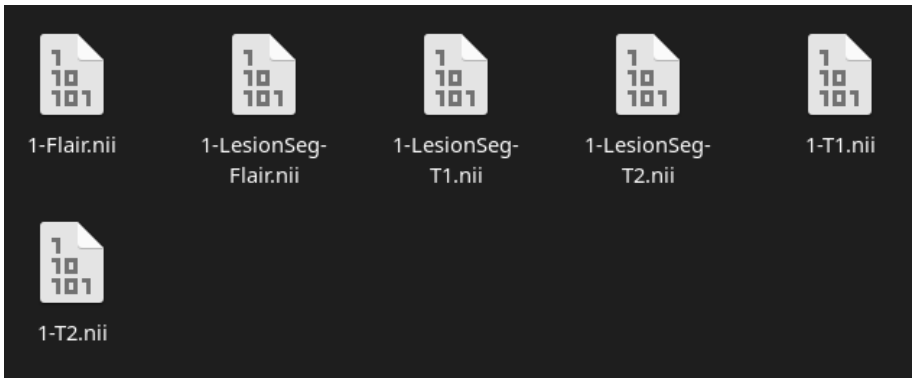


Figure 4.3: Patient 1 Folder

Since we are not storing any patient information, we will move all the **.nii** files into a specific folder named new_dataset for further processing when we select the MRI scans for this project.

After inspecting the folder, we found that each sub-folder is named Patient-X, where X represents the patient ID, ranging from 1 to 60. We decided to examine the folder for Patient 1 to understand its structure. We discovered that each patient has six images: three are the original MRIs, and the other three are segmentations of the original MRIs. The file names are based on the imaging techniques used, such as T1, T2, and FLAIR, along with their corresponding

segmentation files, as shown in Figure 4.3.

Out of these six files, only the three original MRI images are relevant for our use. While the segmentation files could be useful for a more in-depth analysis of lesion layers, we decided to plot the middle layer of the images to observe their differentiation (see Figure 4.4). However, we will focus on the original MRI images for our classification problem.

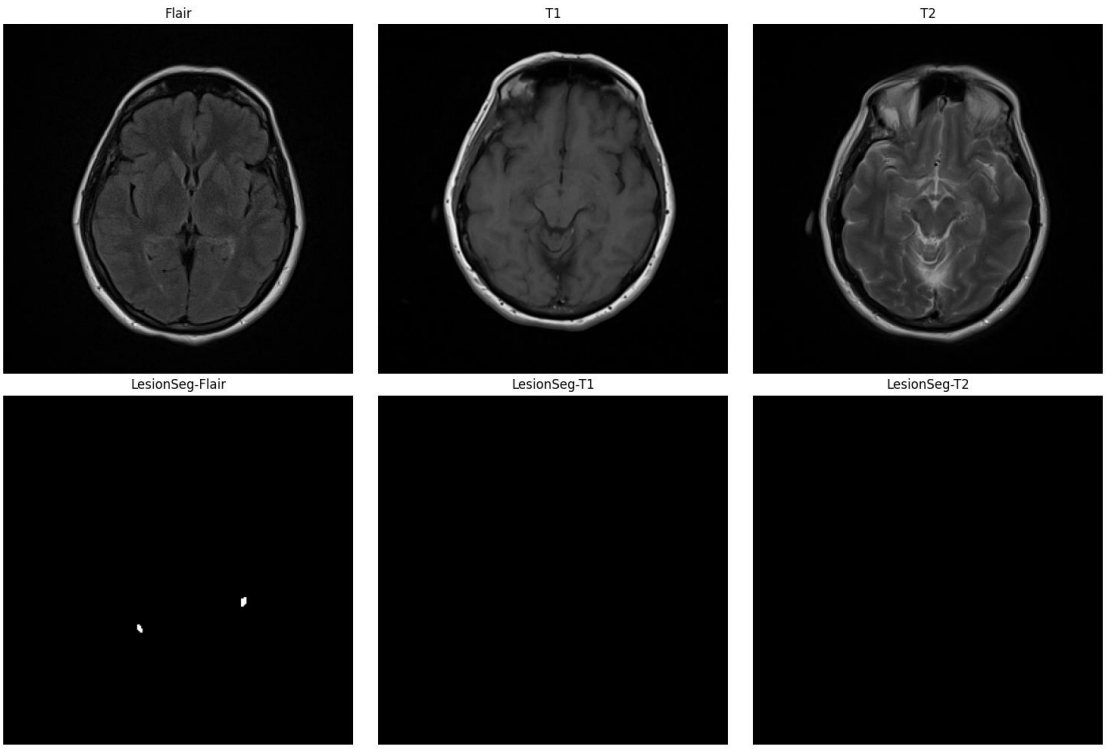


Figure 4.4: Patient 1 Folder Images

To finalize the processing of dataset 1, we created a new script to move the T1, T2, and FLAIR images into separate folders based on their names for each patient, all organized within a folder called **new_dataset**. This was done using a function named **add_to_new_dataset** (see Code A.2). After this process, we now have 60 images per sub-folder in our new folder structure (see Figure 4.5), which can potentially be used for our model.

Figure 4.5: New folder Structure for the New Dataset

4.3.2.2 Dataset 2

In this dataset, we identified two interesting files with the extensions **.md** and **.txt**. The file with the **md** extension contains only a license for using the dataset, while the **txt** file is a README in plain text format, providing a description of the dataset.

The README file indicates that the dataset is organized into multiple folders based on the medical centers where the data originated, specifically "best" and "ljubljana." The "best" folder contains sub-folders for **dev_in**, **eval_in** and **train**, whereas the "ljubljana" folder contains a single sub-folder named **dev_out**. Additionally, the README provides information on the types of MRI modalities included in the dataset, which are as follows:

- **flair**: FLAIR modality input images
- **t1**: T1 modality input images
- **pd**: Proton Density (PD) weighted input images
- **t2**: T2 modality input images
- **t1ce**: T1 contrast enhanced modality (enhancement due to Gadolinium injection)
- **gt**: The 'ground-truth' consensus masks to identify locations of lesion voxels
- **fg_mask**: The foreground masks to identify the brain area (i.e. necessary for calculating error retention curves in only the brain area)

Upon reviewing the folder structure, we decided to focus exclusively on the **best** folder since it contains input images organized into train/dev/eval sub-folders, which suits our needs. This allows us to extract more T1, T2, and FLAIR images from folder **train**, while ignoring additional images present in the dataset that are not required for our analysis. The images in this dataset have been preprocessed, and further details about the preprocessing methods can be found in the README file included with the dataset.

When looking the files inside the folders (see Figure 4.6), we can see the files are compressed with , for this reason we create a script called (see Code for more details) so we can move into the folder and sub-folder Flair, T1 and T2 and extract the MRI and put inside and keep control about what we are moving. We decide to only takes pictures from train so we avoid adding other kind of images that from eval/dev folder that can't be used to train our model and due we can't access to all the dataset we want to avoid any kind of error when moving too much images.

When inspecting the files inside the folders, we observed that the ***.nii** files are compressed with **gz**. To handle this, we created a script named **extract_files.py** (see Code A.3 for more details). This script allows us to navigate into the **train** folder and its sub-folders (Flair, T1, and T2), extract the MRI images, and place them in a **new_dataset** directory while maintaining control over the files being moved. We decided to extract only the images from the train folder to avoid including any images from the eval or dev folders, which are not suitable for training

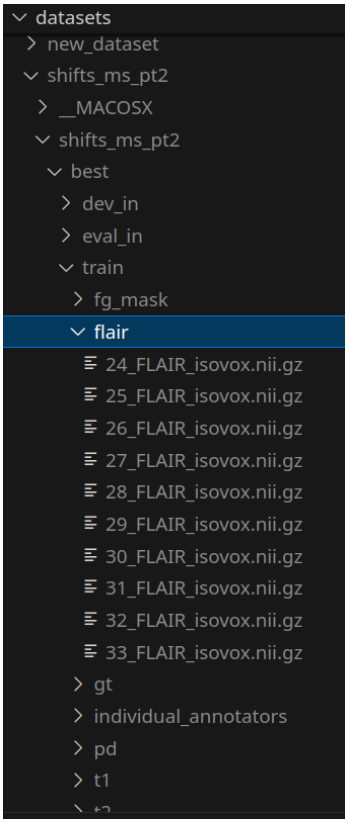


Figure 4.6: Flair Train Folder Structure for Dataset 2

our model. Additionally, since we do not have access to the entire dataset, this approach helps prevent errors that could arise from handling too many images at once.

4.3.2.3 Dataset 3

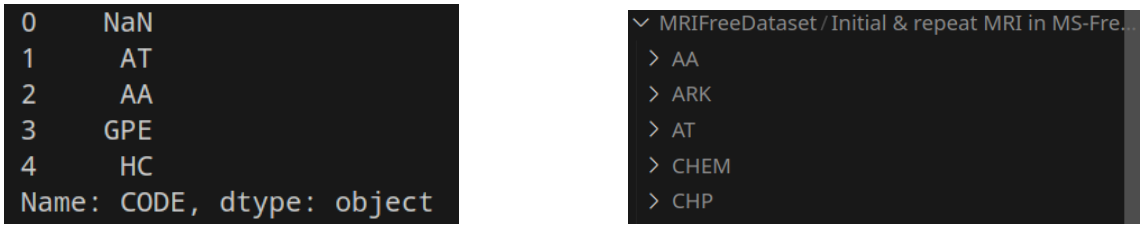
Upon reviewing Dataset 3, we found a **txt** file and an **xls** file, both of which are worth investigating. The **txt** file contains information about the dataset, including its license for use by others, details on the images from the first and second examinations (0-6 months), and lesion segmentation performed with MATLAB, as well as instructions on how to load this data.

The **xls** file contains patient information organized into various columns (see Figure 4.7). Additionally, we noted that each patient has a corresponding exam date and a unique **CODE**, which is represented in the dataset as folder names (see Figure 4.8). Since we are not utilizing any patient-specific information, we have chosen to ignore this data as it is not relevant to our purpose.

```
Index(['N', 'Date Of bitrh ', 'Date of Exam ', 'Age at onset ', 'Unnamed: 4',
       'CODE', 'Unnamed: 6', 'Unnamed: 7'],
      dtype='object')
```

Figure 4.7: Columns from Dataset 3 xls File

As previously mentioned, this dataset contains information from both the first and second



(a) CODE Column from xls File

(b) CODE Folders inside Dataset 3

Figure 4.8: Importance of CODE Column to relate patient with his folder

examinations, where the second exam ranges from 6 to 12 months. Each patient folder contains two sub-folders named "1" and "2," corresponding to the first and second examinations. These sub-folders hold files in various formats, such as **.bmp**, **TIF** and **plq**, which may be found in either folder "1" or "2" without a consistent file pattern. In folder "2," the most common file types are **TIF** and **plq**. Since our current **new_dataset** contains only **nii** files, we decided to exclude this dataset to avoid complications arising from combining different medical image formats within the same project.

4.3.3 Converting MRI to Image

After reviewing Datasets 1, 2, and 3 (see Sections 4.3.2.1, 4.3.2.2, and 4.3.2.3), we created a new folder called **new_dataset** containing the MRI images that we believe are suitable for this project. While we have already organized the images into separate folders based on MRI types (Flair, T1, and T2), we still need to process these images. Since the images are in **nii** format, they need to be converted into a standard image format to be used as input for a neural network.

Neural networks require a valid image format, so we cannot use the **nii** MRI files directly. Therefore, we decided to convert them into **png** images, as this format preserves the details of the grayscale values and offers lossless compression, making it a suitable choice for storing the converted images.

The **nii** format typically contains 3D images, though sometimes it may include more dimensions. To verify the dimensionality of the **nii** images, we created a small script (see Code A.4) to check how many dimensions the images have. After running this script, we found that we have 210 images with three dimensions, meaning we can use all of them without any issues related to dimensionality.

An important aspect of **nii** files is that they are divided into slices. Since we are working with 3D images, each will contain a specific number of slices, which varies by image. For this reason, we will convert each slice into a **png** file, following a standard naming convention: **image_type_x_y**, where "type" represents the MRI type (Flair, T1, or T2), "x" is the index of the **nii** image, and "y" is the slice number within that image. This approach allows us to reassemble the slices later when training the neural network, ensuring that we retain spatial information about the lesions.

You can find the code used for this conversion process in A.5. In this script, we made

several adjustments for convenience. First, we store all the new **png** images in a single folder called **final_dataset**. Additionally, all slices from each **nii** image are stored together in the same sub-folder.

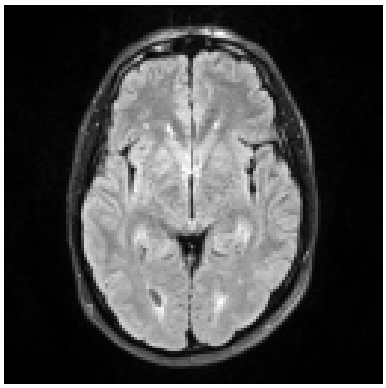
Given the limitations of our computational resources, we also resized the images to 128x128 pixels. Furthermore, we applied normalization to the slices within each image to standardize the pixel intensity values across the dataset.

4.3.4 Selecting MRI with CVS Lesions

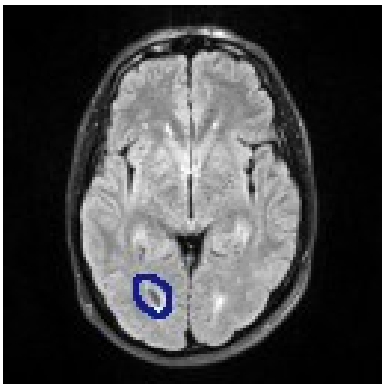
For this process, we manually reviewed all the images in the **final_dataset**. We examined each slice of the original **nii** images to identify lesions, which typically appear brighter (or darker in T1). Lesions can be present in a single slice or across several slices, and their size and shape may vary. Once a lesion was identified, we checked for the presence of a central vein sign (CVS), which appears as a small line or dot within the lesion, representing a vein. The key criterion was to identify a bright lesion with a vein running through the center. Additionally, the geometry of the lesions—often ovoid or round—helped in their identification.

Since this was a manual process conducted by myself, it may introduce some errors into the modeling and final predictions. To minimize this, we carefully analyzed multiple images to select only those that would be useful for our project. The selected images were saved in a folder named **dataset_selected**.

First, we analyzed a Flair image named **image_flair_1_1**, following the naming convention. This image consisted of 19 slices. After examining all the slices, we identified a lesion with a dot (dark area) inside it in slice 8 (see Figure 4.9). Based on the previous criteria, this lesion matched what we were looking for, so we proceeded to select more Flair images with similar lesions.



(a) CVS Lesion on Flair Image 1 - Slice 8



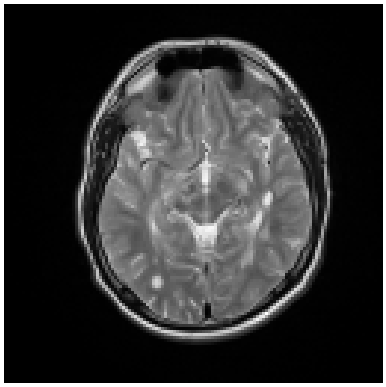
(b) CVS Lesion Marked

Figure 4.9: Lesion in Flair Image marked on Flair

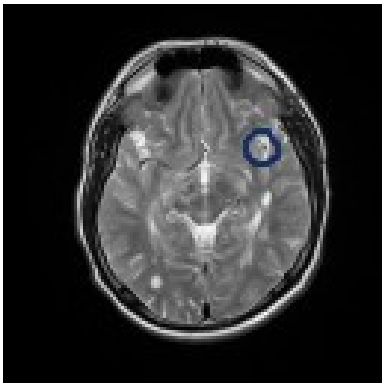
Finding CVS lesions in T1 images was more challenging because T1 lesions appear darker than those in Flair images. Although T1 lesions have the same structure as Flair lesions, they are darker and less prominent. Additionally, due to how T1 images work, lesions do not usually appear as clearly as in Flair images, and in typical studies, they are contrasted with correspond-

ing Flair images. However, in our project, we did not maintain this relationship between Flair and T2. As a result, we were unable to find any T1 images that matched our requirements, so all T1 images were discarded.

For T2 lesions, the identification criteria were similar to those for Flair images. Typically, CVS is detected using both Flair and T2 images, as lesions that appear in Flair should also appear in T2 with some similarities. In Figure 4.10, we can see a possible CVS lesion in slice 16 of a T2 image. This will guide us in selecting more T2 images for the project.



(a) CVS Lesion on T2 Image 9 - Slice 16



(b) CVS Lesion Marked on T2

Figure 4.10: Lesion in T2 Image marked

After filtering the T1, T2, and Flair images from the **final_dataset**, we compiled a new dataset called **dataset_selected** . As shown in Table 4.3 (see Code A.6), we ended up with 5 MRI Flair images containing 110 slices, 0 T1 MRI images, and 8 MRI T2 images containing 189 slices in total.

Based on this data, we noticed a dimensional issue, particularly with the T2 images. Dividing the total number of T2 slices by the number of unique MRIs results in a non-integer value $(189/8 = 23.625)$, whereas the Flair images divide evenly $(110/5 = 22)$. To address this, we will apply padding techniques to ensure all images have the same dimensions when stacking them for our model.

Please note that this is an initial assessment, and the dataset may be further refined or adjusted in the next section.

Type	Unique MRI	Slices
Flair	5	110
T1	0	0
T2	8	189

Table 4.3: MRI Data Selected

4.3.5 Stacking and Padding

As mentioned previously, since our images do not have uniform dimensions to build 3D volumes for our neural network, we need to preprocess them to ensure all volumes have consistent

dimensions. Our approach is slightly more complex than standard padding. We decided to duplicate slices based on the difference between the number of slices in a given volume and the maximum number of slices in our dataset, while maintaining the original order of the slices. This method helps preserve important information by filling gaps with repeated slices instead of introducing noise.

To implement this, we first stack the images based on their filenames to ensure consistency. After that, we apply padding by duplicating slices according to the largest dimensions in our dataset and the difference in the number of slices for each image. This ensures that all 3D volumes have the same shape for input into the neural network.

Since we only have 13 MRI volumes, the idea of using data augmentation was considered. However, data augmentation can alter the original images in various ways, which, although beneficial for expanding the dataset, poses potential issues in medical imaging. These alterations could affect the results of the MRI, leading us to discard data augmentation for this task.

Once the dataset has been preprocessed and all volumes have consistent dimensions, we load the data using **from_tensor_slices**. Due to the dataset's size (13 volumes), we selected a batch size of 2, resulting in only 7 batches. We also applied shuffling to the dataset for randomness.

After completing the preprocessing, we split the data into training, validation, and testing sets. Normally, this would be done using a percentage-based split, but since we know the exact number of volumes and batches, we opted for a manual split. The training set consists of 4 batches (8 MRI volumes), the validation set contains 1 batch (2 MRI volumes), and the test set includes 2 batches (3 MRI volumes).

To avoid repeating the same preprocessing steps, the datasets for training, validation, and testing are saved using TensorFlow's TFRecord format. These files are stored in the **datasets** folder as **train.tfrecord**, **val.tfrecord**, and **test.tfrecord**. The code for this process can be seen in Code A.7.

To verify that the datasets were stored correctly and are usable, we plot one slice from the first volume in each dataset (train, validation, and test), as shown in Figure 4.11. The code for this plot is provided in Code A.9. Additionally, we created a utility file for loading the datasets, as shown in Code A.10.

4.3.6 Addressing Imbalanced Dataset

In our dataset, we have a total of 13 volumes (5 Flair and 8 T2) for positive samples, but this creates an imbalance in the dataset. To address this issue, we decided to select 5 additional Flair and 8 additional T2 MRI volumes from the previously discarded images, creating a balanced dataset. This process was done manually, and the selected images were stored in the **dataset_selected_negative** folder. With this, we are now able to distinguish between lesions that contain CVS and those that do not.

After this step, we confirmed that we now have the same number of Flair and T2 volumes for the negative samples (see Table 4.4). The same code used in Code A.6 was ap-

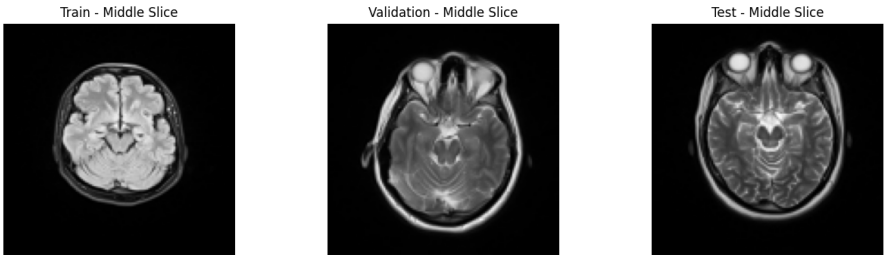


Figure 4.11: Middle Slice from First Volume in each dataset (train, val and test) for Positive Samples

plied here, with the only change being the `dataset_path` variable, which was updated to `dataset_selected_negative`.

Type	Unique MRI	Slices
Flair	5	125
T1	0	0
T2	8	195

Table 4.4: MRI Data Negative Selected

Next, we applied the same process described in Section 4.3.5 to the negative samples. This resulted in three new files: `train_negative.tfrecord`, `val_negative.tfrecord`, and `test_negative.tfrecord`, all with the same dimensions, batch sizes, and number of volumes per batch (see Code A.8). We also plotted one slice from the first volume of each dataset (train, val, and test) to verify that the files were created correctly (see Figure 4.12).

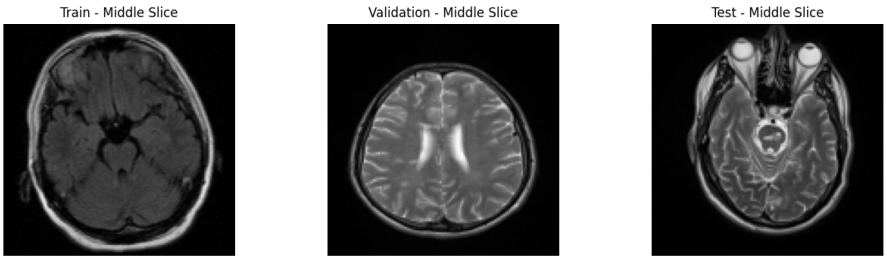


Figure 4.12: Middle Slice from First Volume in each dataset (train, val and test) for Negative Samples

4.3.7 Creating the Final Train/Validation/Test Dataset

In the final step, we decided to combine all the TFRecord files. Since we have separate sets for positive and negative samples, we needed to merge them and assign labels: 1 for samples with CVS and 0 for samples without CVS. To accomplish this, we created a new script called **combined_dataset.py** (see Code A.11). This script allowed us to generate three new files, saved in a new folder named **data**, with the following names: **final_train.tfrecord**, **final_val.tfrecord** and **final_test.tfrecord**. These files are available in the repository. The functions to read this combined dataset along with their corresponding labels can be found in Code A.10.

Additionally, the final dataset consists of 8 batches for training, 2 batches for validation, and 3 batches for testing (execute Code A.12 to print the batches).

4.4 Model Architectures

In this section, we explore three different approaches for detecting CVS lesions in MRI data. The first approach involves a simple CNN model to evaluate its performance with a small dataset. The second approach extends this by implementing a more complex CNN model inspired by CVSNet (see Section 3.7.3). Finally, we apply transfer learning to investigate whether it can enhance performance given the limited size of the dataset.

The dataset used for these models is located in the **data** folder and contains three **tfrecord** files: one for training, one for validation, and one for testing. For the approaches, we set the batch size to 2.

For all models, we focus on three key evaluation metrics: accuracy, F1-score (see function **f1_score** in Code A.13 to know how F1 was calculated), and AUC. Accuracy provides a general sense of prediction correctness, while the F1-score offers a balance between precision and recall, helping us understand both false positives and false negatives. The AUC score measures the model's ability to distinguish between classes.

Additionally, we set a seed value of 42 for TensorFlow and implement the following callbacks: early stopping to prevent overfitting, model check-pointing to save the best model, and learning rate reduction when improvements plateau. All notebooks for these models are available in the repository.

For each model will be used in combination with Keras Tuner to select the best hyperparameters. In our case, Keras Tuner will use Bayesian Optimization to help prevent overfitting, as our dataset is relatively small.

Finally, we decided to conduct two separate training sessions for each model (see Section 5 for training details). The first session will use default parameters, while the second will use the values identified by Keras Tuner. We will then compare the results to determine the best settings for each model type.

4.4.1 Model 1: Basic CNN

The first model (see Figure B.1) consists of two convolutional blocks. The first block contains 32 filters, while the second has 64 filters, both with a kernel size of $(3, 3, 3)$, ReLU activation, and **same** padding. Each block ends with 3D max pooling with a pool size of $(2, 2, 2)$. Following these, a flattening layer is applied, followed by a fully connected layer with 128 neurons and ReLU activation. The output layer is a single neuron with sigmoid activation for binary classification.

Training is performed using the Adam optimizer with default parameters, 50 epochs with binary cross-entropy loss. Full training details are provided in the `models/v1/v1.ipynb` notebook.

4.4.2 Tuning Hyperparameters of Basic CNN

In this step, we decided to adjust our approach and focus on identifying the optimal hyperparameters for our architecture. To do this, we kept the architecture unchanged but used Keras Tuner to select the best hyperparameters based on the data we were using. The layers selected for Keras Tuner are shown in Table 4.5.

Layer	Parameter	Range/Values	Notes
Convolutional #1	Filters	8 to 32, step 8	Kernel, activation, and padding same as original
Convolutional #2	Filters	16 to 64, step 16	Kernel, activation, and padding same as original
Dense #1	Units	32 to 128, step 32	Activation same as original
Adam Optimizer	Learning Rate	$1e-4$ to $1e-2$	Log sampling

Table 4.5: Ranges and settings for each layer in Basic CNN

After running Keras Tuner with 10 trials, which took approximately 43 minutes, we found that the highest validation accuracy achieved was 1. While this initially suggested potential overfitting, we chose to proceed with this configuration for our final training. The final model architecture determined by Keras Tuner is displayed in Figure B.2, , and the full details of the Keras Tuner trials are available in Table 4.6.

Using the hyperparameters selected by Keras Tuner (row 0 in Table 4.6), we then trained our model for 50 epochs with an early stopping patience of 20 epochs. We used the same callbacks as in previous training runs. For a detailed view of the tuning and training process, please refer to the `models/v1/v1_search.ipynb` notebook in the repository.

4.4.3 Model 2: CVSNet-Inspired CNN

The second model (see Figure B.3) draws inspiration from the CVSNet architecture. It includes four convolutional blocks with filter sizes of 32, 64, 128, and 256, respectively, each followed by batch normalization, max pooling, and ReLU activation. Instead of flattening the output from the final convolutional block, a global average pooling layer is applied. This is followed by a fully

	conv1_filters	conv2_filters	dense_units	learning_rate	score
0	16	64	32	0.004947	1.000000
1	24	16	96	0.004845	0.750000
2	24	32	64	0.001920	0.750000
3	24	16	96	0.001425	0.500000
4	24	48	32	0.001078	0.500000
5	32	32	96	0.001848	0.500000
6	8	16	64	0.008634	0.500000
7	32	48	128	0.004975	0.500000
8	32	64	64	0.003033	0.500000
9	24	64	128	0.005224	0.500000

Table 4.6: Information about Keras Tuner run for Basic CNN

connected layer with 512 and 216 units, with dropout rates of 0.2 and 0.4, respectively, between the layers. The model concludes with a single output layer for binary classification.

For training, the learning rate of the Adam optimizer is set to 0.0001, and training is limited to 25 epochs, as the previous training converged at epoch 11. Details of the training process can be found in the **models/v2/v2.ipynb** notebook.

4.4.4 Tuning Hyperparameters of CVSNet-Inspired CNN

Similar to the previous model, we opted to explore another set of hyperparameters while keeping the same model architecture. Using Keras Tuner, we aimed to identify the optimal values for our small dataset. The layers selected for tuning are shown in Table 4.7.

Layer	**Parameter**	**Range/Values**	**Notes**
Convolutional #1	Filters	16 to 32, step 8	Kernel, activation, and padding same as original
Convolutional #2	Filters	32 to 64, step 16	Kernel, activation, and padding same as original
Convolutional #3	Filters	64 to 128, step 32	Kernel, activation, and padding same as original
Convolutional #4	Filters	128 to 256, step 64	Kernel, activation, and padding same as original
Dense #1	Units	16 to 48, step 16	Activation same as original
Dense #2	Units	4 to 12, step 4	Activation same as original
Adam Optimizer	Learning Rate	$1e-4$ to $1e-2$	Log sampling

Table 4.7: Ranges and settings for each layer in CVSNet-Inspired CNN

To determine the best hyperparameters for the second model, we conducted 10 trials, a process that took a total of 1 hour. The highest validation accuracy achieved was 0.5. Notably, this accuracy value was consistent across all trials (see Table 4.8). The final model selected by Keras Tuner is depicted in Figure B.4.

From Table 4.8 we chose the hyperparameters for training, setting the model to run for 25

	conv1_filters	conv2_filters	conv3_filters	conv4_filters	dense_units	dense_units_2	learning_rate	score
0	32	32	64	192	32	8	0.004864	0.500000
1	24	32	64	128	48	12	0.004782	0.500000
2	24	32	96	192	16	12	0.000112	0.500000
3	32	32	128	192	48	8	0.007121	0.500000
4	32	64	64	128	32	4	0.000202	0.500000
5	32	64	64	128	48	8	0.003366	0.500000
6	16	48	128	128	32	8	0.000133	0.500000
7	32	64	128	128	16	8	0.000140	0.500000
8	32	32	128	192	16	12	0.002597	0.500000
9	32	32	96	128	16	4	0.002953	0.500000

Table 4.8: Information about Keras Tuner run for CVSNet-Inspired CNN

epochs with early stopping enabled at a patience of 5. Other callbacks remain the same as in the previous run. For a more detailed view of the training process, refer to the notebook `models/v2/v2_search.ipynb` in the repository.

4.4.5 Model 3: Transfer Learning with ResNet50

The third approach initially aimed to use a pre-trained 3D model for medical images but was adapted to combine a pre-trained 2D model (ResNet50) with a custom 3D CNN. Since ResNet50 is a well-known model for 2D images (with weights pre-trained on **imagenet**), we incorporate it to process individual slices of our 3D MRI data.

Given that the dataset is 3D, we employ a Lambda layer to process each slice independently. Each slice, being grayscale, is expanded into 3 channels by duplicating the grayscale value. This allows us to leverage the power of the 2D pre-trained ResNet50 model within a 3D CNN framework.

Following the base ResNet50 model, we build a top model with a single convolutional block containing 32 filters, a $(3, 3, 3)$ kernel size, batch normalization, ReLU activation, max pooling, and dropout to prevent overfitting. The fully connected layer contains 16 neurons, and the final output layer has 1 neuron for binary classification (see Figure B.5).

More details about the model architecture and training process are available in the **models/v3/v3.ipynb** notebook.

4.4.6 Tuning Hyperparameters of Transfer Learning with ResNet50

To complete the architecture we started, we will use Keras Tuner to optimize the hyperparameters for this final model. The main challenge with this model was the Lambda layer (responsible for using ResNet50). Since Keras Tuner does not support tuning this type of layer, we focused on optimizing only the top layers. Given the computational complexity of this model, largely due to the Lambda layer, we narrowed the search range for the remaining layers. The values selected for tuning are listed in Table 4.9.

As this approach required more computational resources than previous models, we reduced the number of trials to 5 instead of 10. Despite fewer trials, the process took 1 hour and 13 minutes, with three runs encountering memory issues on our machine. For this reason, we

Layer	Parameter	Range/Values	Notes
Convolutional	Filters	2 to 4, step 2	Kernel, activation, and padding same as original
Dense	Units	4 to 8, step 8	Activation same as original
Adam Optimizer	Learning Rate	$1e-4$ to $1e-2$	Log sampling

Table 4.9: Ranges and settings for each layer in Transfer Learning with ResNet50

decided to limit the process to 5 trials. The best result achieved a validation accuracy of 0.75, while the other trials reached 0.5 (see Table 4.10). The final model selected was the first one from the table, as shown in Figure B.6.

	conv1_filters	dense_units	learning_rate	score
0	2	8	0.005189	0.750000
1	4	4	0.002112	0.500000
2	4	8	0.001307	0.500000
3	4	8	0.002852	0.500000
4	2	4	0.003188	0.500000

Table 4.10: Information about Keras Tuner run for Transfer Learning with ResNet50

Finally, we also adjusted the training parameters similarly, using 25 epochs with an early stopping patience of 10. Like the non-Keras Tuner model, this one converged at epoch 11. Details of the training process can be found in the notebook `models/v3/v3_search.ipynb` in the repository.

Training Results and Discussion

5

5.1 Training Results

In this section, we will discuss the results of the training phase for the models outlined in Sections 4.4.1, 4.4.2, 4.4.3, 4.4.4, 4.4.5 and 4.4.6. We will analyze how the techniques used and the dataset (particularly small datasets) affect the performance of the models, regardless of the complexity of the architectures. Additionally, we will explore the challenges posed by small datasets and how limited computational resources impacted the training process.

It is important to note that we have three types of architectures: Basic CNN (Section 4.4.1), CVSNet-Inspired CNN (Section 4.4.3), and Transfer Learning with ResNet50 (Section 4.4.5), where the layer values were set manually. Additionally, we created three models with the same architectures but with some layer values optimized using Keras Tuner. The training for these models was conducted using the tuned values (Sections 4.4.2, 4.4.4, and 4.4.6, respectively), allowing us to compare the performance of manually crafted models against those optimized with Keras Tuner.

5.1.1 Training the Basic CNN

The training of the basic CNN model was the smoothest in terms of epochs. Although it was the simplest model, it had the highest number of trainable parameters ($67,165,377$ with 256.22MB memory used), which made the training more exhaustive. As mentioned in Section 4.4, we used various callbacks during training, with early stopping being the most important to prevent unnecessary epochs without improvements in validation accuracy.

Initially, we set 50 epochs for training with batch sizes of 2. However, the early stopping callback halted the training at epoch 11, as no further improvement in validation accuracy was observed. We also reduced the learning rate to prevent overfitting, although this had a minimal effect given our dataset size.

The metrics (Figure 5.1) display the accuracy, loss, F1 score, and AUC for both the training and validation datasets. As shown in the plots, there was significant overfitting, especially in the validation set (red curve), where the values remained relatively unchanged. This suggests that the model was "guessing" rather than making accurate predictions, likely due to the small dataset or an overly complex network. Despite our efforts to design a small CNN, the model still had a large number of trainable parameters.

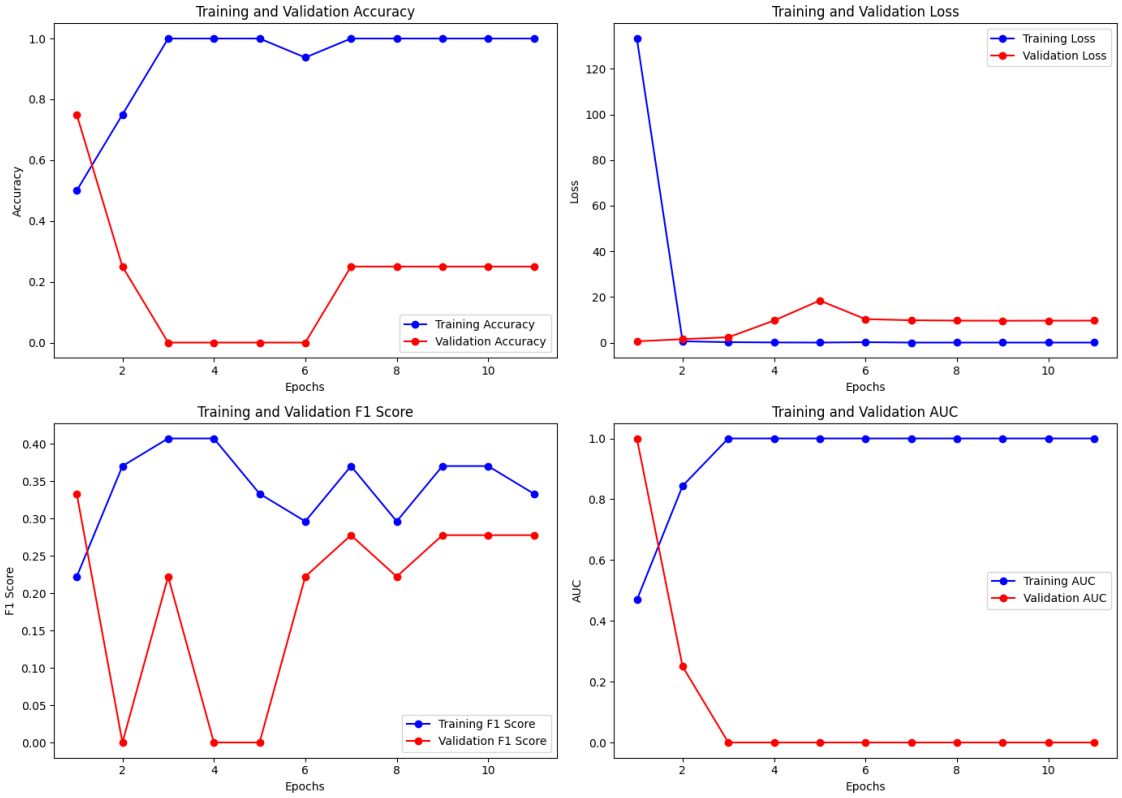


Figure 5.1: Basic CNN Metrics

The F1 score plot shows a relatively smooth pattern, which is a positive sign. However, the same overfitting issue appears in the validation set. Therefore, it was unnecessary to test this model on the test dataset, as we anticipated poor results based on the training phase.

5.1.2 Training the Keras Tuner Basic CNN

Similar to the hand-crafted training (Section 5.1.1) the number of trainable parameters was higher than the craft-handed version ($16,805,441$ with 64.11MB of memory used) which is a reduction of trainable parameters and memory used near $\approx 75\%$ which talking in computational resources is a huge step to avoid training too complex models with no directions that are computational expensive. At the training phase we used the same 50 epochs with the same batch size of 2.

For the callbacks used they were the same, specially the early stopping has the same patience which was 10, but in this tuning version this stop occurs at epoch 21 (10 more epochs from the hand-crafted version), besides we were able to train for more epochs and also reducing the computational resources used for this model, the final metrics results in a similar way, since the begin we can see a clearly overfitting in the validation accuracy.

For the metrics (Figure 5.2) we can see that in validation for the accuracy, loss, F1 and AUC curves were mostly static indicating a clear overfitting (similar to the hand-crafted version) since the beginning, something similar occurs for the trainings curves, in that case that occurs

for accuracy, loss and AUC but not to the F1 curve, in this case for training the curve has a oscillated pattern which can indicate the learning rate used was too high or the batch size were small (which is our case).

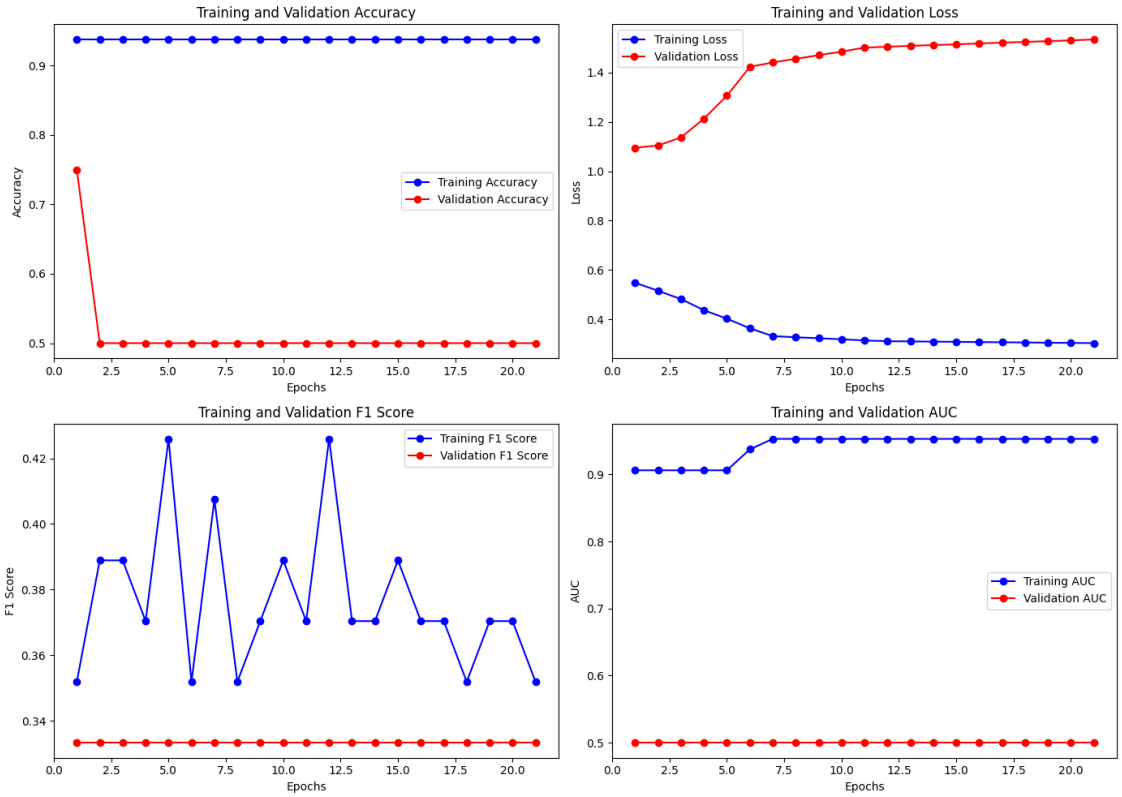


Figure 5.2: Basic CNN Metrics with Tuning

5.1.3 Training CVSNet-Inspired CNN

In this second approach, we built a CNN inspired by the CVSNet architecture, with modifications to address overfitting issues. This network was more complex than the one described in Section 4.4.1 and included mechanisms to reduce overfitting, such as dropout, batch normalization, and max pooling. Despite the increased complexity, the number of trainable parameters was lower ($1,426,689$ with 5.44MB of memory used) because most of the new layers were designed to prevent overfitting.

Recognizing that 50 epochs might be excessive, we reduced the training to 25 epochs, using the same early stopping criteria. Once again, early stopping halted the training at epoch 6 due to the lack of improvement in the selected metrics.

The metrics (Figure 5.3) reveal the same overfitting problem observed in the previous approach. Across all metrics (Accuracy, F1, and AUC), the validation curves remained relatively static with minor fluctuations, indicating that the model was not learning effectively from the dataset. As with the first model, we chose not to apply the saved model for testing, knowing it would produce poor results.

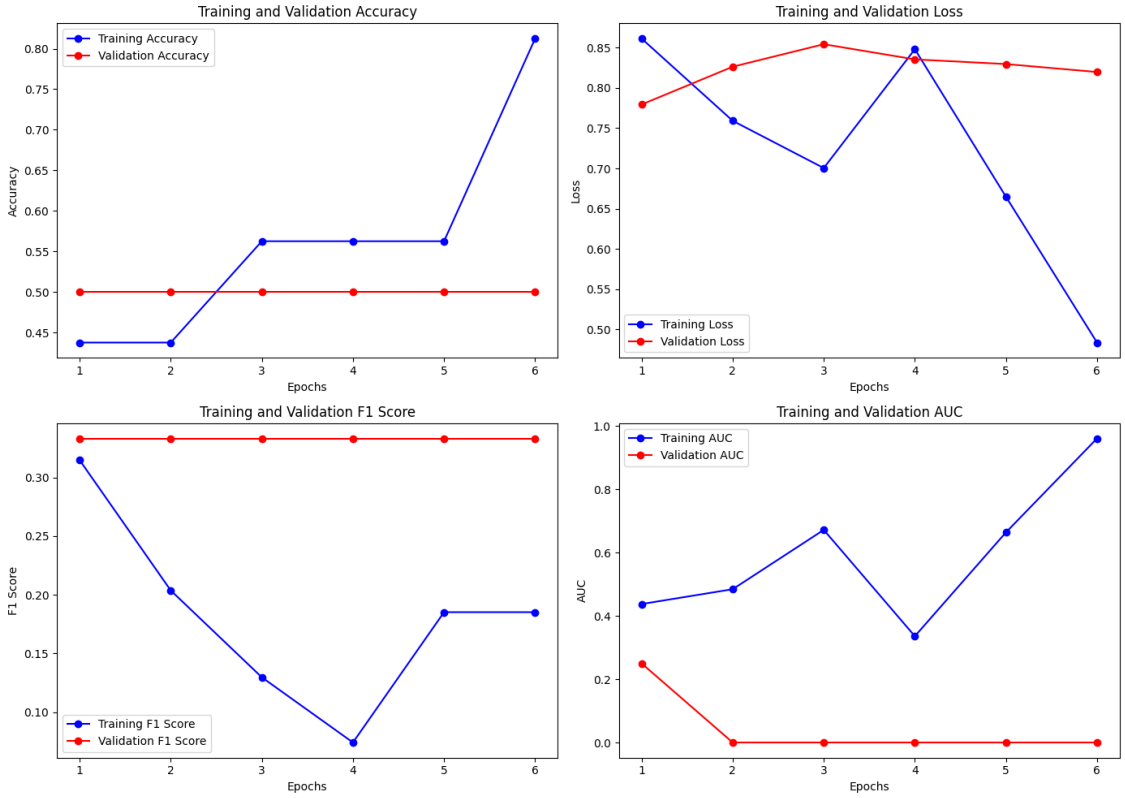


Figure 5.3: CVSNet-Inspired CNN Metrics

5.1.4 Training the Keras Tuner CVSNet-Inspired CNN

In this Tuner version, using the training described in Section 5.1.3, we observe a reduction in the number of trainable parameters ($422,993$ with 1.61MB of memory used). This represents a decrease of approximately $\approx 70.4\%$ in both parameters and memory usage, making our model significantly lighter than the hand-crafted version. For training, we used 25 epochs with early stopping applied, set with a patience of 5.

Although we achieved reductions in parameter count and memory usage, the effective number of usable epochs remained the same in both versions, with training stopping at epoch 6. A notable difference is that, in the hand-crafted version, we encountered memory allocation issues at the start of training, whereas this new version avoids such problems. However, there was no improvement in validation accuracy, with both versions stabilizing around 50% from the outset.

Regarding the metrics (see Figure 5.4), the training accuracy in this Tuner version was lower across epochs compared to the hand-crafted version. This suggests that with fewer hyperparameters, the model found it more challenging to learn effectively. Nonetheless, despite the lower training accuracy, validation accuracy remained the same across both versions, indicating that the limitation may lie not in the number of trainable parameters but rather in the small batch sizes and limited dataset quality.

On the other hand, the loss curves reveal a rapid drop in validation loss while remaining

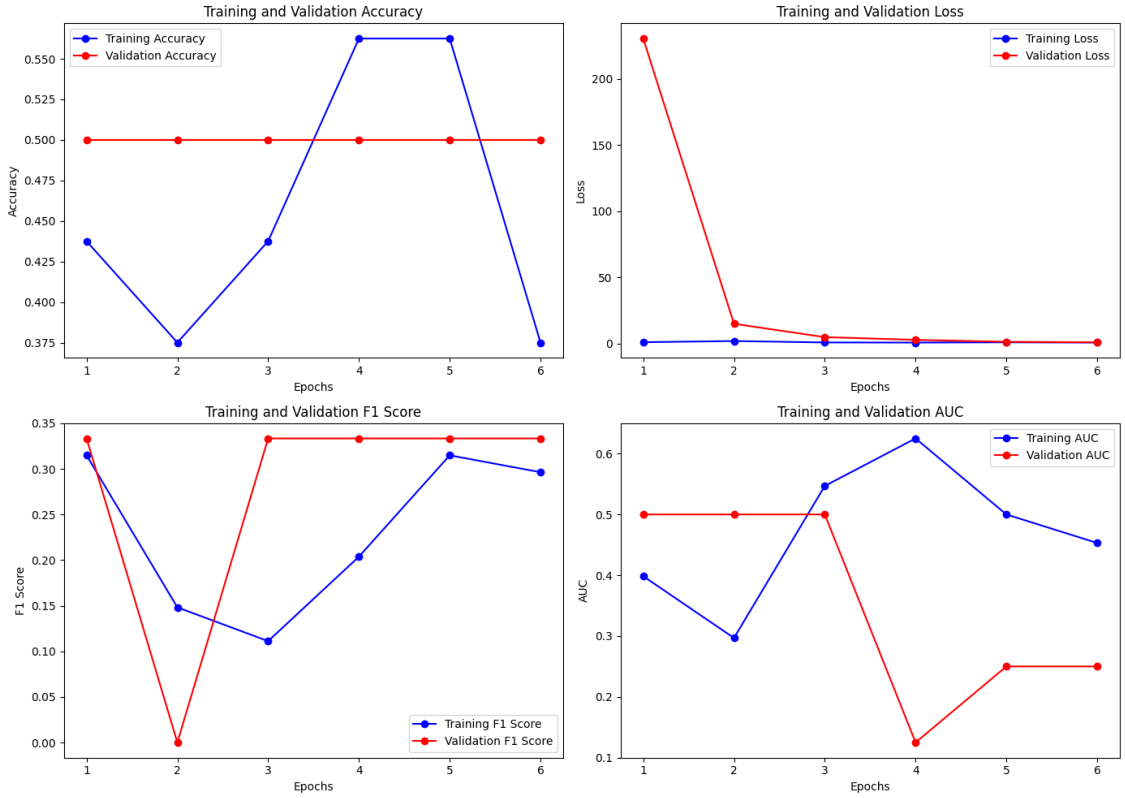


Figure 5.4: CVSNet-Inspired CNN Metrics with Tuning

steady during training, suggesting a potential convergence issue where the model may have become stuck. Additionally, the F1 score curve shows more consistency between training and validation, though some peaks in validation hint at issues likely related to batch size. Finally, the AUC curve exhibits a similar pattern to the F1 score.

5.1.5 Training Transfer Learning with ResNet50

For the final approach, we combined transfer learning (explained in Section 4.4.5) with a simple CNN. Due to the use of transfer learning with a lambda layer, this model required significant computational resources for this layer, while the CNN itself had relatively few trainable parameters ($1,770,113$ with 6.75MB of memory used, slightly more than the model described in Section 4.4.3). To mitigate overfitting, we employed the same techniques used in the previous models, and the number of trainable parameters was limited due to our computational constraints.

Similar to the previous approach, we used 25 epochs and the same early stopping criteria, which halted the training at epoch 8. The metrics (Figure 5.5) show a slightly better performance than the previous models, but overfitting remained an issue. The training accuracy curve shows improvement over the epochs, while the validation accuracy plateaued after a certain point.

The F1 score plot indicates that, while the training curve improved, the validation curve remained static, similar to the previous models. For this reason, we also decided not to test this model, as we anticipated poor results based on the training performance.

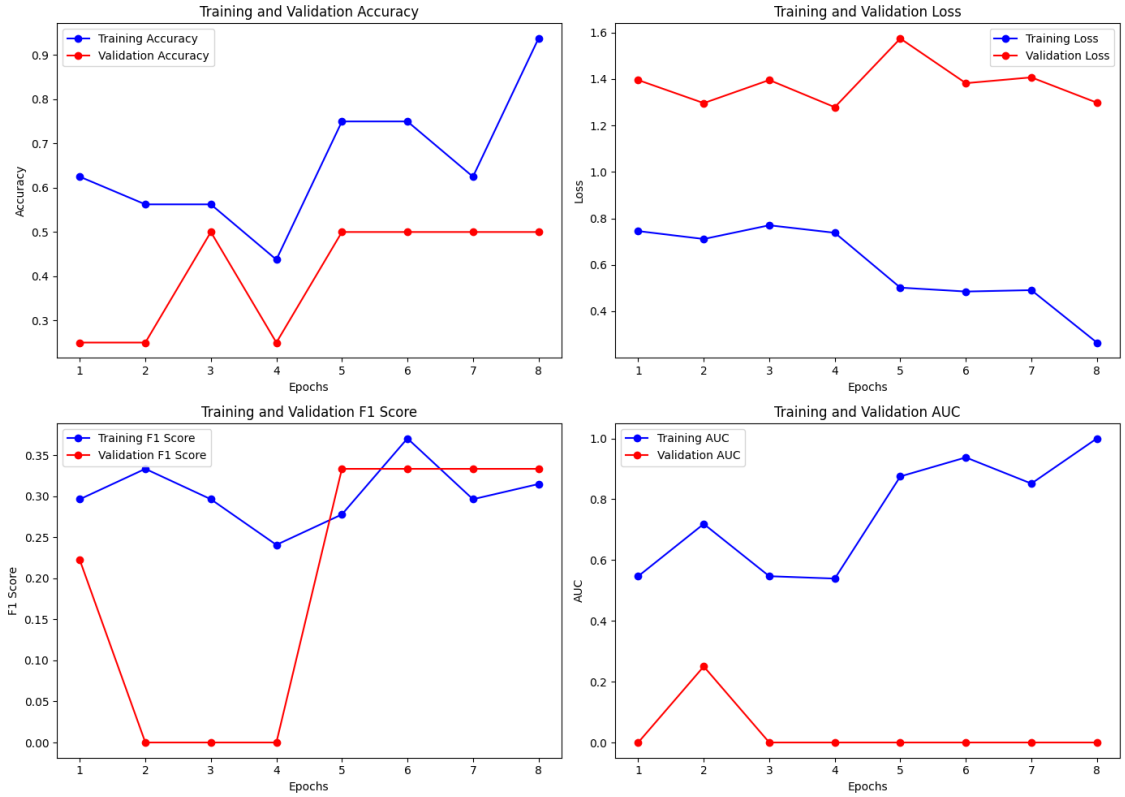


Figure 5.5: Transfer Learning With ResNet50 Metrics

5.1.6 Training the Keras Tuner Transfer Learning with ResNet50

In this approach, we observe a substantial reduction in trainable parameters $(110,631$ with 432.15KB of memory used), with a reduction of $\approx 94\%$. However, this cannot be viewed solely as an improvement, as the tuning was performed on an architecture incorporating a transfer learning layer (specifically, a Lambda layer), which is computationally expensive. Furthermore, Keras Tuner does not support tuning this type of layer, so it was excluded from the tuning process. To manage memory constraints, we also had to reduce the number of tuning attempts, as each iteration involves executing this Lambda layer.

Despite reducing the number of attempts and excluding the Lambda layer from tuning, this approach continued to face challenges. Tuning only two layers with Keras Tuner still posed difficulties because we were working with a high parameter count. Consequently, we had to restrict the range of possible configurations for testing, which limited the tuning options. Thus, while we achieved a reduction in trainable parameters, this was primarily due to computational resource limitations rather than tuning optimizations alone.

To address these limitations, we kept the training at 25 epochs but increased the patience for early stopping from 5 to 10, given that the model would be lighter than the hand-crafted version. With this adjustment, training extended to 11 epochs (compared to 9 in the hand-crafted version), but validation accuracy remained at 0.50 across epochs, indicating signs of overfitting despite the parameter reduction.

The metrics shown in Figure 5.6 reveal that the training accuracy curve has peaks and drops, suggesting suboptimal batch sizes and data quality. Meanwhile, the validation loss remains consistently higher than training loss, pointing to an underfitting issue. This underfitting suggests that the model is overly simple and unable to capture essential patterns, and the Lambda layer's transfer learning does not alleviate this. The F1 score and AUC curves further confirm that the model is not effectively learning, primarily due to the constrained hyperparameter search imposed by computational limits.

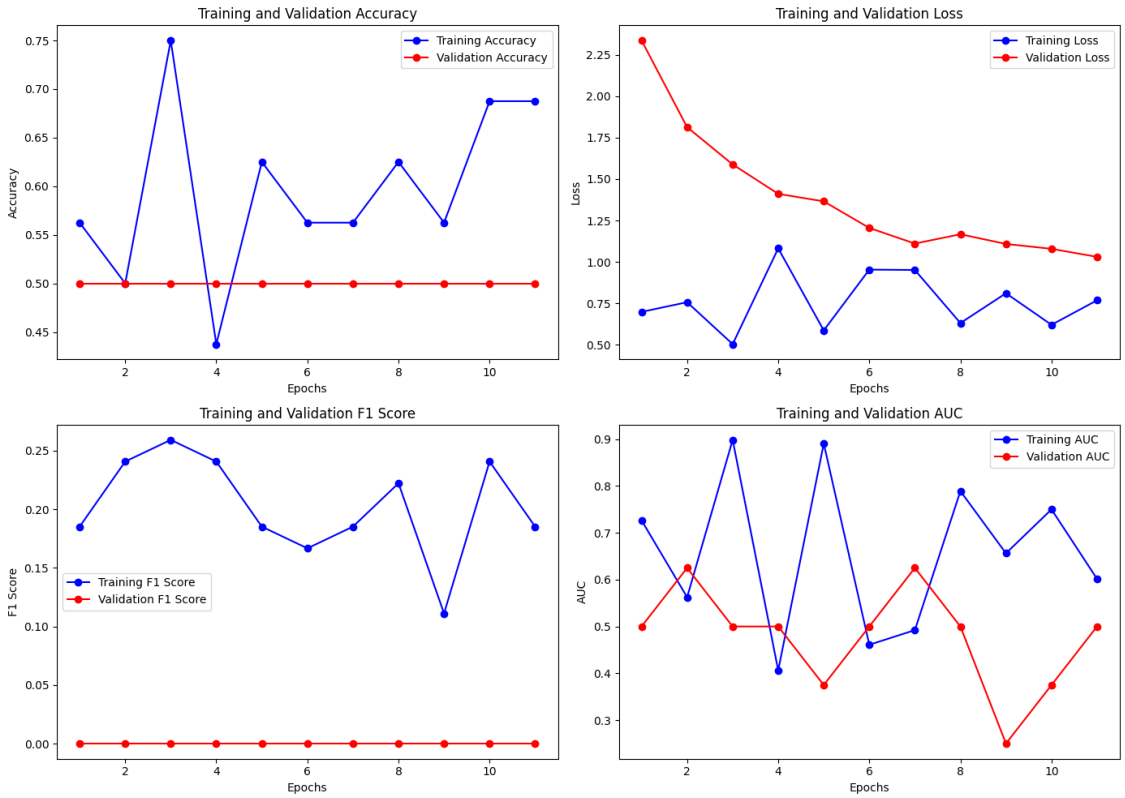


Figure 5.6: Transfer Learning With ResNet50 Metrics with Tuning

5.2 Discussion

As observed in the training phase, none of the models yielded optimal results. This can be attributed to several factors. First, as mentioned earlier, the dataset was specifically designed to detect CVS lesions in MRI images, but we were only able to obtain 13 valid positive MRI scans and 13 negative scans. The small dataset led to issues during training, with the model often running out of data during some epochs. This caused the model to "learn too fast" from individual batches, raising an error.

In addition to the dataset size, we faced computational resource limitations, particularly in the transfer learning model. We had to significantly reduce the number of layers in the 3D-CNN top model, as more complex architectures caused the system to crash due to insufficient resources. This likely contributed to the suboptimal training outcomes.

Another challenge arose during the dataset preparation. We needed to convert 3D MRI images into PNG files by slicing the volumes. When creating the final dataset, we had to adjust the images to ensure uniform dimensions across all volumes. However, our approach—duplicating slices to match the maximum slice count—may not have been the best solution, as it likely introduced noise, which contributed to the poor results observed throughout this project.

Despite the challenges encountered, we decided to tune the hyperparameters in each architecture to see if we could improve our results. This approach allowed us to reduce computing resources, although the results did not change considerably and led to faster overfitting (or underfitting in the case of the transfer learning approach with tuning). As shown in the various metrics from this training phase, most curves indicate similar issues related to data quality, batch size, and, in some cases, overly simple models that struggle to extract useful information from the dataset.

In conclusion, we can observe that factors such as dataset quality, limited dataset size, small batch sizes, and the complexity or simplicity of models can significantly impact the training phase, regardless of the techniques used to mitigate overfitting or underfitting. Another key consideration is the trade-off between data quality and model architecture. This trade-off can influence results independently of specific techniques; for instance, in the transfer learning approach with tuning, we had to reduce model complexity due to computational limitations, which led to poorer results compared to a more complex, hand-crafted model.

Given the poor training results, testing our models on the test split was deemed unnecessary, as we could predict in advance that the outcomes would not be satisfactory.

Conclusions

6

The initial goal of this project was to develop a neural network capable of detecting CVS lesions in MRI scans to identify Multiple Sclerosis (MS). However, this task proved to be more challenging than expected. The primary difficulty arose when attempting to find a suitable dataset focused on CVS lesion detection in MRI related to MS. Given the specificity of this topic, finding a dedicated dataset was highly difficult. As a result, a manual process was required to combine various MS-related datasets into a single dataset that could potentially identify CVS lesions in MRI scans.

Before creating a dataset for our specific purpose, it was essential to first understand multiple sclerosis (MS), how it is diagnosed, and the techniques used for diagnosis, particularly MRI. We discovered that the McDonald criteria is the most well-known method for diagnosing MS, as it incorporates a range of diagnostic techniques and continues to evolve over time. Additionally, it was necessary to understand the Central Vein Sign (CVS) and its significance in diagnosis.

The Central Vein Sign plays a crucial role in distinguishing MS from other diseases. Although it can appear in other conditions, it is more commonly associated with MS. Another important step was reviewing the state of the art; we found several papers highlighting the importance of CVS in MS diagnosis and its potential as a future diagnostic biomarker alongside others. We also observed a growing number of approaches that integrate CVS with MS detection and artificial intelligence (AI), yielding progressively better results. However, sensitivity remains a challenge, especially in distinguishing MS from similar diseases.

Although we were able to locate several datasets that seemed appropriate at first glance, we soon discovered that CVS lesions were not commonly represented. Additionally, it was challenging to determine when a CVS lesion was present, especially without external expert guidance. For this reason, the dataset created for this project may not be an ideal representation of what is needed. Ultimately, we identified only 13 MRI scans that, based on our criteria, showed CVS lesions.

With the final dataset in place, we proposed three different model architectures but two variants each, described in Section 4.4. We conducted multiple training experiments to determine the best parameters and hyperparameters for each model. However, due to the small size of our dataset, the results were not as promising or appropriate for the goals we set out to achieve.

Although we were unable to create an effective model architecture suited for small datasets, we gained valuable insights into the current state of the art in this field. Additionally, we explored methods for padding MRI slices to ensure uniform dimensions across all data, which was es-

sential when the original images had varying dimensions. Another key aspect of our project was the use of transfer learning. We combined 2D transfer learning models with a 3D-CNN via a lambda layer to enhance the results (see Section 4.4.5, 5.1.5), which demonstrated a promising approach to harnessing the power of 2D transfer learning in 3D CNN architectures.

On the other hand, we found that using Keras Tuner was an important step to improve model performance while also reducing the computational resources needed for training, yet maintaining similar results. This approach allowed us to reduce the number of trainable parameters in two of the three proposed model architectures without significant information loss, as both models achieved comparable final results. However, in the case of transfer learning, the tuning approach performed worse than the original, likely due to the complexity required to effectively explore hyperparameters in a standard tuning process.

In conclusion, this project underscores the complexities of working with medical images, especially MRI scans, and the challenges presented by the scarcity of specialized datasets for niche problems. Despite the difficulties of independently addressing such a task, we identified several promising strategies for managing small datasets, such as hyperparameter tuning, applying transfer learning with a lambda layer, and using cloning padding to augment the dataset effectively. These approaches represent progress toward addressing these issues, particularly when data is limited.

Challenges and Future Directions

7

7.1 Challenges

Throughout this project, we encountered several challenges that limited our results and the scope of the study. These challenges included:

1. **Lack of Specific Open Datasets**: There were no publicly available datasets that suited our needs. The problem of detecting CVS lesions in MS is highly specialized within the neurology field, and we could not find any open datasets specifically focused on CVS lesions in MS.

2. **Absence of Expert Guidance**: Since this project was conducted independently, we did not have access to expert assistance in identifying which MRIs contained CVS lesions. As a result, the dataset we generated is likely subject to noise, which could negatively impact the final results.

3. **Heterogeneous Image Dimensions**: The dataset was assembled manually from multiple open datasets, which led to inconsistencies in image dimensions. To address this, we had to apply techniques to standardize the dimensions without losing important information or introducing noise.

4. **Limited Computational Resources**: Training neural networks, especially those working with 3D images, is computationally expensive. Since we were not affiliated with any institution and relied solely on a laptop for computational power, this limitation affected the scope and quality of the results.

7.2 Future Directions

Machine learning and deep learning are excellent tools for working with medical images to aid in disease identification and pattern recognition. This is particularly relevant in the study of diseases like multiple sclerosis, which is difficult to diagnose and for which the underlying causes remain unclear. Based on the challenges we faced, future research could focus on the following directions:

1. **Collaboration with Experts**: Continue the project with the help of experts, preferably from an established institute, to gain access to more MRI data and ensure the dataset is cleaner and more uniform in terms of dimensions.

2. **Collaboration with CVSNet Authors**: Reach out to the authors of CVSNet 3.7.3 to explore potential collaboration and expand on their research, with the goal of developing a better tool for diagnosing CVS lesions in MRI.

3. **Utilization of Pre-Trained Models**: Explore the use of pre-trained models specifically designed for medical images. These models could be fine-tuned using transfer learning to improve the performance of the neural network on our specific task.

4. **Expanding the Dataset**: Use a dataset that includes not only MS-related images but also images from other diseases. This would allow the model to better differentiate between MS cases based on CVS biomarkers, rather than focusing solely on CVS in MRI scans specific to MS.

5. **Combining Transfer Learning Techniques**: Experiment with combining two types of transfer learning using a lambda layer—one for feature extraction from 2D images and another for 3D image extraction. This approach could improve the model's ability to learn and generalize better from medical images.

Python Code

A.1 Dataset Analyze Code

Code A.1: Python function to analyze dataset folders

```python
import os
from collections import defaultdict

def analyze_dataset_folders(root_folder):
    """
    Analyze the dataset folder so we can check how many uniques file formats
    we have and the number of elements, also we summarize the number of
        elements by extension
    and a breakdown the subfolder dataset based on their format included

    :root_folder (str) -> String format of the root folder

    """
    extension_count = defaultdict(int)
    extension_folders = defaultdict(lambda: defaultdict(int))  # To count files
         per root folder per extension
    subfolder_count = defaultdict(int)  # To count the number of subfolders in
        each root folder

    for dirpath, _, filenames in os.walk(root_folder):
        for filename in filenames:
            # Extract the file extension
            ext = os.path.splitext(filename)[1]
            if ext:
                # Find the root folder
                root_subfolder = os.path.join(root_folder, os.path.relpath(
                    dirpath, root_folder).split(os.sep)[0])
                extension_count[ext] += 1
                extension_folders[root_subfolder][ext] += 1

        # Count the subfolders in the root folder
        root_subfolder = os.path.join(root_folder, os.path.relpath(dirpath,
            root_folder).split(os.sep)[0])
        subfolder_count[root_subfolder] = len([d for d in os.listdir(
            root_subfolder) if os.path.isdir(os.path.join(root_subfolder, d))])
```

```
    print(f"Number of unique file formats: {len(extension_count)}")
    total_files = sum(extension_count.values())
    print(f"Total number of elements: {total_files}\n")

    print("File Extensions and Total Counts:")
    for ext, count in extension_count.items():
        print(f"Extension: {ext}, Total Count: {count}")

    print("\nDetailed Breakdown by Root Folder:")
    for root_folder, extensions in extension_folders.items():
        print(f"\nRoot Folder: {root_folder}:")
        for ext, count in extensions.items():
            print(f"  Extension {ext}: Count: {count}")
        print(f"  Subfolders: {subfolder_count[root_folder]} subfolders")

if __name__ == "__main__":
    root_folder = "datasets/"
    analyze_dataset_folders(root_folder)
```

A.2 Add Dataset 1 to new Dataset

Code A.2: Python function to add mri images to new dataset

```
import os
import shutil

def organize_files_from_all_patients(root_folder, new_dataset_folder):
    """
    Organizes NIfTI files from all patient subfolders into Flair, T1, and T2
        subfolders inside new_dataset,
    ignoring files with 'Segmentation' or 'LesionSeg' in their names.

    :param root_folder: Path to the root folder containing all patient folders.
    :param new_dataset_folder: Path to the new dataset folder where files will
        be organized.
    """
    subfolders = ['Flair', 'T1', 'T2']

    for subfolder in subfolders:
        os.makedirs(os.path.join(new_dataset_folder, subfolder), exist_ok=True)

    for patient_folder in os.listdir(root_folder):
        patient_path = os.path.join(root_folder, patient_folder)

        if os.path.isdir(patient_path):
            print(f"Processing {patient_folder}...")

            for file_name in os.listdir(patient_path):
```

```
                if 'LesionSeg' in file_name:
                    continue

                if 'Flair' in file_name:
                    target_folder = os.path.join(new_dataset_folder, 'Flair')
                elif 'T1' in file_name:
                    target_folder = os.path.join(new_dataset_folder, 'T1')
                elif 'T2' in file_name:
                    target_folder = os.path.join(new_dataset_folder, 'T2')
                else:
                    print(filename)
                    continue

                source_path = os.path.join(patient_path, file_name)
                target_path = os.path.join(target_folder, file_name)
                shutil.copy(source_path, target_path)
                print(f"Copied {file_name} to {target_folder}")

root_folder = os.path.join("datasets",
    "Brain MRI Dataset of Multiple Sclerosis with Consensus Manual Lesion
        Segmentation and Patient Meta Information")

new_dataset_folder = os.path.join("datasets", "new_dataset")

organize_files_from_all_patients(root_folder, new_dataset_folder)
```

A.3 Extract and Move from Dataset 2

Code A.3: Python function to extract nii files from gz

```
import os
import gzip

# Define root folders and subdirectories
root_folder = "datasets"
dataset_2 = os.path.join(root_folder, "shifts_ms_pt2")
best_folder = os.path.join(dataset_2, "shifts_ms_pt2/best/train")
folders_to_keep = ['flair', 't1', 't2']
new_folder = "datasets/new_dataset"

def extract_gz_file(gz_path, output_path):
    """
    Extracts the contents of a .gz file and saves it to the specified output
        path.

    :param gz_path: Path to the .gz file.
    :param output_path: Path where the extracted .nii file will be saved.
    :return: None
    """
```

```
    # Open the .gz file in read-binary mode and extract the content
    with gzip.open(gz_path, 'rb') as f_in:
        with open(output_path, 'wb') as f_out:
            f_out.write(f_in.read())

def extract_and_move():
    """
    Loops through the specified image folders, extracts the .gz files,
    and moves the extracted .nii files to a new folder.

    :param root_folder: Path to the root folder containing all patient folders.
    :param new_dataset_folder: Path to the new dataset folder where files will
        be organized.
    :return: None
    """
    for image_folder in folders_to_keep:
        print(f"Processing folder {image_folder}")

        move_to = os.path.join(new_folder, image_folder.capitalize())

        source_folder = os.path.join(best_folder, image_folder)

        for image in os.listdir(source_folder):
            if image.endswith('.gz'):
                image_path = os.path.join(source_folder, image)

                nii_file_name = os.path.splitext(image)[0]

                output_path = os.path.join(move_to, nii_file_name)

                # Extract the .gz file and save the result in the destination
                    folder
                extract_gz_file(image_path, output_path)
                print(f"Extracted {image} to {output_path}")

if __name__ == '__main__':
    extract_and_move()
```

A.4 Count Dimension for New Dataset

Code A.4: Python function to analyze dataset folders

```
import os
import nibabel as nib
from collections import defaultdict

def count_files_by_dimension(root_folder, new_dataset_folder):
    """
```

```
    Count the number of files in each dimensionality (2D, 3D, 4D, etc.) for the
        given dataset folders.

    :param root_folder: Path to the root folder containing all patient folders.
    :param new_dataset_folder: Path to the new dataset folder where files will
       be checked.
    :return: None
    """
    dimension_type_counts = defaultdict(int)

    base_dir = os.path.join(root_folder, new_dataset_folder)
    folders = ['Flair', 'T1', 'T2']

    for folder in folders:
        folder_path = os.path.join(base_dir, folder)
        for file in os.listdir(folder_path):
            if file.endswith('.nii') or file.endswith('.nii.gz'):
                file_path = os.path.join(folder_path, file)
                # Load the NIfTI image
                img = nib.load(file_path)
                img_shape = img.shape
                # Determine the dimensionality (2D, 3D, 4D, etc.)
                dimension_type = len(img_shape)
                # Increment the count for the corresponding dimension type
                dimension_type_counts[dimension_type] += 1

    for dimension_type, count in sorted(dimension_type_counts.items()):
        print(f"Files with {dimension_type}D -> {count}")

if __name__ == '__main__':
    root_folder = 'datasets'
    new_dataset_folder = 'new_dataset'

    count_files_by_dimension(root_folder, new_dataset_folder)
```

A.5 Convert and Resize nii to png

Code A.5: Python function to convert and resize nii to png

```
import os
import nibabel as nib
import numpy as np
import matplotlib.pyplot as plt
from skimage.transform import resize

def convert_nii_to_png(input_dir, output_dir='datasets/final_dataset',
    target_size=(128, 128)):
    """
```

```
    Converts .nii images into PNG slices, stores them in a single folder with a
         specified naming convention,
    and resizes the images for further processing.

    Args:
        input_dir (str): The input directory containing subfolders Flair, T1,
            T2 with .nii files.
        output_dir (str): The directory where the PNG images will be stored (
            default is 'final_dataset').
        target_size (tuple): The target size for resizing the images (default
            is (128, 128).

    Returns:
        None
    """

    subfolders = ['Flair', 'T1', 'T2']
    output_subfolders = ['flair', 't1', 't2']

    if not os.path.exists(output_dir):
        os.makedirs(output_dir)

    for subfolder, output_subfolder in zip(subfolders, output_subfolders):
        print(f"Processing {subfolder} images")
        folder_path = os.path.join(input_dir, subfolder)

        nii_file_number = 1  # Keeps track of the .nii file number (x in the
            naming format)

        for file in os.listdir(folder_path):
            if file.endswith('.nii'):
                print(f"Processing slides from nii image #{nii_file_number}")
                nii_path = os.path.join(folder_path, file)
                img = nib.load(nii_path)
                img_data = img.get_fdata()

                # Loop through the slices
                for slice_idx in range(img_data.shape[2]):
                    slice_img = img_data[:, :, slice_idx]

                    # Normalize the slice
                    slice_img = (slice_img - np.min(slice_img)) / (np.max(
                        slice_img) - np.min(slice_img))
                    slice_img = (slice_img * 255).astype(np.uint8)

                    # Resize the image
                    slice_img_resized = resize(slice_img, target_size,
                        anti_aliasing=True)

                    # Format image_type_x_y.png
```

```
                    output_filename = f"image_{output_subfolder}_{
                        nii_file_number}_{slice_idx + 1}.png"
                    output_path = os.path.join(output_dir, output_filename)

                    plt.imsave(output_path, slice_img_resized, cmap='gray')

                nii_file_number += 1
    print("Processing Done")

if __name__ == '__main__':
    input_directory = 'datasets/new_dataset'
    convert_nii_to_png(input_directory, target_size=(128, 128))
```

A.6 Count Unique MRI Images

Code A.6: Python function to count unique MRI based on the type

```
import os
import pandas as pd

def analyze_mri_images(dataset_path):
    """
    Get number of the unique MRI files stored in the path

    The images are expected to follow this format: image_{type}_{index}_{slice
        }.png
    where:
    - type: flair, t1, t2 (the type of MRI scan)
    - index: a unique index for each MRI image
    - slice: the slice number of the image

    :param dataset_path: Folder name containing the MRI image slices in PNG
        format
    :return: DataFrame with columns 'type', 'unique MRI', and 'slices'
    """
    mri_data = {
        'flair': {'unique_indexes': set(), 'total_slices': 0},
        't1': {'unique_indexes': set(), 'total_slices': 0},
        't2': {'unique_indexes': set(), 'total_slices': 0}
    }

    for filename in os.listdir(dataset_path):
        if filename.endswith('.png'):
            parts = filename.split('_')  # Split filename by '_'
            if len(parts) == 4:
                image_type = parts[1]  # flair, t1, or t2
                image_index = parts[2]  # index of the MRI image

                # Add the index to the dictionary based on type
```

```
                if image_type in mri_data:
                    mri_data[image_type]['unique_indexes'].add(image_index)
                    mri_data[image_type]['total_slices'] += 1

    data = {
        'type': list(mri_data.keys()),
        'unique MRI': [len(mri_data[image_type]['unique_indexes']) for
            image_type in mri_data],
        'slices': [mri_data[image_type]['total_slices'] for image_type in
            mri_data]
    }

    return pd.DataFrame(data)

if __name__ == '__main__':
    dataset_path = 'datasets/dataset_selected'
    df = analyze_mri_images(dataset_path)
    print(df)
```

A.7 Stacking and Padding

Code A.7: Python function to stack png files into 3D Volumes for positive

```
import os
import numpy as np
import tensorflow as tf
from PIL import Image

def read_image(filepath):
    img = Image.open(filepath).convert('L')  # Convert to grayscale
    img = np.array(img)
    return img

# Function to stack slices and create 3D volumes
def stack_slices(image_folder):
    volumes = {}
    for filename in sorted(os.listdir(image_folder)):
        if filename.endswith('.png'):
            # Extract type, index, and slice number from filename
            parts = filename.split('_')
            img_type, img_index, img_slice = parts[1], parts[2], parts[3].split
                ('.')[0]
            img_index = int(img_index)
            img_slice = int(img_slice)

            # Create a new volume key combining img_type and img_index
            volume_key = f"{img_type}_{img_index}"

            # Create a new volume if the combination of type and index is new
```

```
            if volume_key not in volumes:
                volumes[volume_key] = []

            filepath = os.path.join(image_folder, filename)

            img = read_image(filepath)
            volumes[volume_key].append((img_slice, img))

    for key in volumes:
        # Sort by slice number
        volumes[key] = np.stack([img for _, img in sorted(volumes[key], key=
            lambda x: x[0])])

    return volumes

def pad_volumes(volumes):
    # Find the maximum number of slices across all volumes
    max_slices = max(vol.shape[0] for vol in volumes.values())

    padded_volumes = {}

    for key, vol in volumes.items():
        current_slices = vol.shape[0]

        if current_slices == max_slices:
            # No need to pad if the volume already has the max number of slices
            padded_volumes[key] = vol
            continue

        # Calculate how many slices need to be added
        missing_slices = max_slices - current_slices

        # Proportional duplication: Calculate how many times each slice should
            be duplicated
        duplication_factor = max_slices // current_slices
        remaining_slices = max_slices % current_slices

        new_slices = []

        for i in range(current_slices):
            new_slices.append(vol[i])

            for _ in range(duplication_factor - 1):
                new_slices.append(vol[i])

            if remaining_slices > 0:
                new_slices.append(vol[i])
                remaining_slices -= 1

        new_slices = new_slices[:max_slices]
```

```
        padded_vol = np.stack(new_slices, axis=0)
        padded_volumes[key] = padded_vol

    return padded_volumes

def create_tf_dataset(volumes, batch_size):
    volume_list = [tf.convert_to_tensor(vol, dtype=tf.float32) for vol in
        volumes.values()]

    dataset = tf.data.Dataset.from_tensor_slices(volume_list)

    dataset = dataset.map(lambda vol: tf.convert_to_tensor(vol, dtype=tf.
        float32))

    dataset = dataset.batch(batch_size)

    return dataset

def process_images(image_folder, batch_size):
    # Stack the slices into volumes
    volumes = stack_slices(image_folder)
    # Pad the volumes to ensure they have the same dimensions
    padded_volumes = pad_volumes(volumes)
    # Create tensorflow dataset
    dataset = create_tf_dataset(padded_volumes, batch_size=batch_size)
    return dataset

def split_dataset(dataset):
    # Total number of batches in the dataset
    total_batches = dataset.cardinality().numpy()

    train_batches = 4
    val_batches = 1
    test_batches = total_batches - train_batches - val_batches

    train_dataset = dataset.take(min(train_batches, total_batches))

    remaining_dataset = dataset.skip(train_batches)

    val_dataset = remaining_dataset.take(min(val_batches, total_batches -
        train_batches))

    remaining_dataset = remaining_dataset.skip(val_batches)

    test_dataset = remaining_dataset.take(min(test_batches, total_batches -
        train_batches - val_batches))

    return train_dataset, val_dataset, test_dataset
```

```
# Function to serialize a single example
def serialize_example(volume):
    volume_tensor = tf.io.serialize_tensor(volume)
    feature = {
        'volume': tf.train.Feature(bytes_list=tf.train.BytesList(value=[
            volume_tensor.numpy()]))
    }
    example_proto = tf.train.Example(features=tf.train.Features(feature=feature
        ))
    return example_proto.SerializeToString()

# Function to save dataset to TFRecord
def write_tfrecord(dataset, file_path):
    with tf.io.TFRecordWriter(file_path) as writer:
        for batch in dataset:  # Iterate over batches in the dataset
            for volume in batch:  # Each volume in the batch
                serialized_volume = serialize_example(volume)
                writer.write(serialized_volume)


if __name__ == '__main__':
    image_folder = 'datasets/dataset_selected'
    dataset = process_images(image_folder, batch_size=2)

    train, val, test = split_dataset(dataset)

    print(f"Train dataset cardinality: {train.cardinality().numpy()}")
    print(f"Test dataset cardinality: {val.cardinality().numpy()}")
    print(f"Val dataset cardinality: {test.cardinality().numpy()}")

    write_tfrecord(train, 'datasets/train.tfrecord')
    write_tfrecord(val, 'datasets/val.tfrecord')
    write_tfrecord(test, 'datasets/test.tfrecord')

    print(f"Train dataset cardinality: {train.cardinality().numpy()}")
    print(f"Test dataset cardinality: {val.cardinality().numpy()}")
    print(f"Val dataset cardinality: {test.cardinality().numpy()}")
```

A.8 Stacking and Padding for Negative

Code A.8: Python function to stack png files into 3D Volumes for negative

```
from stacking_and_padding import process_images, split_dataset, write_tfrecord

if __name__ == '__main__':
    image_folder = 'datasets/dataset_selected_negative'
    dataset = process_images(image_folder, batch_size=2)

    train, val, test = split_dataset(dataset)
```

```
    print(f"Train dataset cardinality: {train.cardinality().numpy()}")
    print(f"Test dataset cardinality: {val.cardinality().numpy()}")
    print(f"Val dataset cardinality: {test.cardinality().numpy()}")

    write_tfrecord(train, 'datasets/train_negative.tfrecord')
    write_tfrecord(val, 'datasets/val_negative.tfrecord')
    write_tfrecord(test, 'datasets/test_negative.tfrecord')

    print(f"Train dataset cardinality: {train.cardinality().numpy()}")
    print(f"Test dataset cardinality: {val.cardinality().numpy()}")
    print(f"Val dataset cardinality: {test.cardinality().numpy()}")

    print("Datasets saved as TFRecord files in datasets folder")
```

A.9 Load and Show

Code A.9: Python function to verify TFRecord files and plot

```
import matplotlib.pyplot as plt
from utils import load_tfrecord

# Function to visualize the middle slice from a 3D volume
def get_middle_slice(volume):
    middle_index = volume.shape[0] // 2  # Get the middle slice index
    return volume[middle_index]

# Function to visualize one volume from a dataset
def visualize_middle_slice_from_splits(train_dataset, val_dataset, test_dataset
    ):
    # Initialize the plot for 3 subplots
    fig, axs = plt.subplots(1, 3, figsize=(15, 5))

    # Extract from Train
    for batch in train_dataset.take(1):
        train_volume = batch[0].numpy()
        middle_slice_train = get_middle_slice(train_volume)
        axs[0].imshow(middle_slice_train, cmap='gray')
        axs[0].set_title("Train - Middle Slice")
        axs[0].axis('off')
        break

    # Extract from Val
    for batch in val_dataset.take(1):
        val_volume = batch[0].numpy()
        middle_slice_val = get_middle_slice(val_volume)
        axs[1].imshow(middle_slice_val, cmap='gray')
        axs[1].set_title("Validation - Middle Slice")
        axs[1].axis('off')
```

```
        break

    # Extract from Test
    for batch in test_dataset.take(1):
        test_volume = batch[0].numpy()
        middle_slice_test = get_middle_slice(test_volume)
        axs[2].imshow(middle_slice_test, cmap='gray')
        axs[2].set_title("Test - Middle Slice")
        axs[2].axis('off')
        break

    plt.subplots_adjust(wspace=0.4, hspace=0.4)
    plt.savefig("datasets/load_show.png")
    plt.show()

if __name__ == '__main__':
    train = load_tfrecord('datasets/train.tfrecord').batch(2)
    val = load_tfrecord('datasets/val.tfrecord').batch(2)
    test = load_tfrecord('datasets/test.tfrecord').batch(2)

    visualize_middle_slice_from_splits(train, val, test)
```

A.10 Utils

Code A.10: Python function to verify TFRecord files and plot

```
import tensorflow as tf

def parse_tfrecord_fn(example_proto):
    feature_description = {
        'volume': tf.io.FixedLenFeature([], tf.string)
    }
    parsed_features = tf.io.parse_single_example(example_proto,
        feature_description)
    volume = tf.io.parse_tensor(parsed_features['volume'], out_type=tf.float32)
    return volume

# Function to load dataset from TFRecord file
def load_tfrecord(file_path):
    raw_dataset = tf.data.TFRecordDataset(file_path)
    parsed_dataset = raw_dataset.map(parse_tfrecord_fn)
    return parsed_dataset

def parse_combined_tfrecord_fn(example_proto, add_channel=False):
    feature_description = {
        'volume': tf.io.FixedLenFeature([], tf.string),
        'label': tf.io.FixedLenFeature([], tf.int64)
    }
```

```
    parsed_features = tf.io.parse_single_example(example_proto,
        feature_description)

    # Parse the volume tensor and the label
    volume = tf.io.parse_tensor(parsed_features['volume'], out_type=tf.float32)
    if add_channel:
        volume = tf.expand_dims(volume, axis=-1)
        volume.set_shape([33, 128, 128, 1])

    label = parsed_features['label']
    return volume, label

# Function to load the combined TFRecord files
def load_combined_tfrecord(file_path, batch_size=2, add_channel=False,shuffle=
    False,buffer_size=20):
    raw_dataset = tf.data.TFRecordDataset(file_path)

    parsed_dataset = raw_dataset.map(lambda x: parse_combined_tfrecord_fn(x,
        add_channel=add_channel))

    if shuffle:
        parsed_dataset = parsed_dataset.shuffle(buffer_size=buffer_size)

    parsed_dataset = parsed_dataset.batch(batch_size)

    return parsed_dataset

def count_records(dataset):
    return sum(1 for _ in dataset)
```

A.11 Combined Datasets

Code A.11: Python function to join full dataset

```
import tensorflow as tf
from utils import parse_tfrecord_fn, count_records

def load_tfrecord_with_label(file_path):
    label = 0 if 'negative' in file_path else 1

    raw_dataset = tf.data.TFRecordDataset(file_path)
    parsed_dataset = raw_dataset.map(parse_tfrecord_fn)

    # Add label to the volume
    labeled_dataset = parsed_dataset.map(lambda volume: (volume, label))
    return labeled_dataset

def combine_datasets(positive_file, negative_file, buffer_size=8, batch_size=2)
    :
```

```
    # Load datasets with labels
    positive_dataset = load_tfrecord_with_label(positive_file)
    negative_dataset = load_tfrecord_with_label(negative_file)

    # Combine the positive and negative datasets
    combined_dataset = positive_dataset.concatenate(negative_dataset)

    # Shuffle and batch the dataset
    combined_dataset = combined_dataset.batch(batch_size)
    return combined_dataset

def serialize_example(volume, label):
    volume_tensor = tf.io.serialize_tensor(volume)
    label_int = int(label)
    feature = {
        'volume': tf.train.Feature(bytes_list=tf.train.BytesList(value=[
            volume_tensor.numpy()])),
        'label': tf.train.Feature(int64_list=tf.train.Int64List(value=[
            label_int]))
    }
    example_proto = tf.train.Example(features=tf.train.Features(feature=feature
        ))
    return example_proto.SerializeToString()

def write_combined_tfrecord(dataset, file_path):
    with tf.io.TFRecordWriter(file_path) as writer:
        for batch_volumes, batch_labels in dataset:
            for volume, label in zip(batch_volumes, batch_labels):
                serialized_example = serialize_example(volume, label)
                writer.write(serialized_example)

if __name__ == '__main__':

    train_dataset_combined = combine_datasets(
        'datasets/train.tfrecord',
        'datasets/train_negative.tfrecord')
    val_dataset_combined = combine_datasets(
        'datasets/val.tfrecord',
        'datasets/val_negative.tfrecord')
    test_dataset_combined = combine_datasets(
        'datasets/test.tfrecord',
        'datasets/test_negative.tfrecord')

    write_combined_tfrecord(
        train_dataset_combined,
        'data/final_train.tfrecord')
    write_combined_tfrecord(
        val_dataset_combined,
        'data/final_val.tfrecord')
    write_combined_tfrecord(
```

```
        test_dataset_combined,
        'data/final_test.tfrecord')

    print("Final datasets saved as TFRecord files.")
```

A.12 Check Combined Batches

Code A.12: Python function to check number of batches in datasets

```
from utils import load_combined_tfrecord, count_records

def print_batches(final_dataset):
    print(f"Total Batches: {count_records(final_dataset)}")
    for batch_num, (volume, label) in enumerate(final_dataset):
        print(f"Batch {batch_num + 1}:")
        print(f"Volume shape: {volume.shape}")
        print(f"Labels: {label.numpy()}")

final_train_dataset = load_combined_tfrecord('data/final_train.tfrecord')
final_val_dataset = load_combined_tfrecord('data/final_val.tfrecord')
final_test_dataset = load_combined_tfrecord('data/final_test.tfrecord')

print("Train data:\n")
print_batches(final_train_dataset)
print("\nValidation data:\n")
print_batches(final_val_dataset)
print("\nTest data:\n")
print_batches(final_test_dataset)
```

A.13 Model Utils

Code A.13: Python functions Models Utils

```
import sys
import os
sys.path.append('/home/kurose/Desktop/master/viu_master_thesis')

from scripts.dataset_selected.utils import parse_combined_tfrecord_fn,
    load_combined_tfrecord, count_records
from tensorflow.keras.utils import plot_model
import tensorflow as tf
import matplotlib.pyplot as plt

current_dir = os.path.dirname(os.path.abspath(__file__))
base_path = os.path.dirname(current_dir)

train_file_path = os.path.join(base_path, 'data', 'final_train.tfrecord')
```

```
val_file_path = os.path.join(base_path, 'data', 'final_val.tfrecord')
test_file_path = os.path.join(base_path, 'data', 'final_test.tfrecord')

def load_dataset(batch_size = 2):
    # Load train, validation, and test datasets

    train_dataset = load_combined_tfrecord(train_file_path,batch_size=
        batch_size, add_channel=True,shuffle=True,buffer_size=20)
    val_dataset = load_combined_tfrecord(val_file_path, batch_size=batch_size,
        add_channel=True,shuffle=True,buffer_size=20)
    test_dataset = load_combined_tfrecord(test_file_path, batch_size=batch_size
        , add_channel=True,shuffle=True,buffer_size=20)
    return train_dataset, val_dataset, test_dataset

def get_input_shape(train):
    for i in train:
        input_shape = i[0].shape[1:]
        break
    return input_shape

def f1_score(y_true, y_pred):
    # Cast y_true to float32 to match y_pred type
    y_true = tf.cast(y_true, tf.float32)

    y_pred = tf.round(y_pred)

    # Calculate True Positives, False Positives, and False Negatives
    tp = tf.reduce_sum(tf.cast(y_true * y_pred, 'float32'), axis=0)
    fp = tf.reduce_sum(tf.cast((1 - y_true) * y_pred, 'float32'), axis=0)
    fn = tf.reduce_sum(tf.cast(y_true * (1 - y_pred), 'float32'), axis=0)

    # Precision and Recall
    precision = tp / (tp + fp + tf.keras.backend.epsilon())
    recall = tp / (tp + fn + tf.keras.backend.epsilon())

    # F1 Score
    f1 = 2 * (precision * recall) / (precision + recall + tf.keras.backend.
        epsilon())
    # Taking the mean over the batch
    f1 = tf.reduce_mean(f1)

    return f1

def save_and_print_model(model, file_path="model_image.png"):
    model.summary()
    plot_model(model, to_file=file_path, show_shapes=True, show_layer_names=
        True)
    print(f"Model architecture saved to {file_path}")

def get_callbacks(
```

```
    checkpoint_path='best_model.h5',
    monitor_metric='val_loss',
    patience=10,
    reduce_factor=0.1,
    reduce_patience=5,
    reduce_min_lr=1e-6):

    # Early Stopping
    early_stopping = tf.keras.callbacks.EarlyStopping(
        monitor=monitor_metric,
        patience=patience,
        restore_best_weights=True
    )

    # Model Checkpoint
    model_checkpoint = tf.keras.callbacks.ModelCheckpoint(
        filepath=checkpoint_path,
        monitor=monitor_metric,
        save_best_only=True,
        save_weights_only=True
    )

    # Reduce LR On Plateau
    reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(
        monitor=monitor_metric,
        factor=reduce_factor,
        patience=reduce_patience,
        min_lr=reduce_min_lr
    )

    return [early_stopping, model_checkpoint, reduce_lr]

def get_metrics():
    return ['accuracy', tf.keras.metrics.AUC(name='auc'), f1_score]

def plot_metrics(history, save_path=None):
    epochs = range(1, len(history.history['loss']) + 1)

    fig, axs = plt.subplots(2, 2, figsize=(14, 10))

    # Top-left plot: Accuracy and Validation Accuracy
    axs[0, 0].plot(epochs, history.history['accuracy'], 'bo-', label='Training
        Accuracy')
    axs[0, 0].plot(epochs, history.history['val_accuracy'], 'ro-', label='
        Validation Accuracy')
    axs[0, 0].set_title('Training and Validation Accuracy')
    axs[0, 0].set_xlabel('Epochs')
    axs[0, 0].set_ylabel('Accuracy')
    axs[0, 0].legend()
```

```
    # Top-right plot: Loss and Validation Loss
    axs[0, 1].plot(epochs, history.history['loss'], 'bo-', label='Training Loss
        ')
    axs[0, 1].plot(epochs, history.history['val_loss'], 'ro-', label='
        Validation Loss')
    axs[0, 1].set_title('Training and Validation Loss')
    axs[0, 1].set_xlabel('Epochs')
    axs[0, 1].set_ylabel('Loss')
    axs[0, 1].legend()

    # Bottom-left plot: F1 Score and Validation F1 Score
    axs[1, 0].plot(epochs, history.history['f1_score'], 'bo-', label='Training
        F1 Score')
    axs[1, 0].plot(epochs, history.history['val_f1_score'], 'ro-', label='
        Validation F1 Score')
    axs[1, 0].set_title('Training and Validation F1 Score')
    axs[1, 0].set_xlabel('Epochs')
    axs[1, 0].set_ylabel('F1 Score')
    axs[1, 0].legend()

    # Bottom-right plot: AUC and Validation AUC
    axs[1, 1].plot(epochs, history.history['auc'], 'bo-', label='Training AUC')
    axs[1, 1].plot(epochs, history.history['val_auc'], 'ro-', label='Validation
         AUC')
    axs[1, 1].set_title('Training and Validation AUC')
    axs[1, 1].set_xlabel('Epochs')
    axs[1, 1].set_ylabel('AUC')
    axs[1, 1].legend()

    plt.tight_layout()

    if save_path:
        plt.savefig(save_path)
        print(f"Plot saved to {save_path}")

    plt.show()
```

Model Architectures

B

B.1 Model 1: Basic CNN

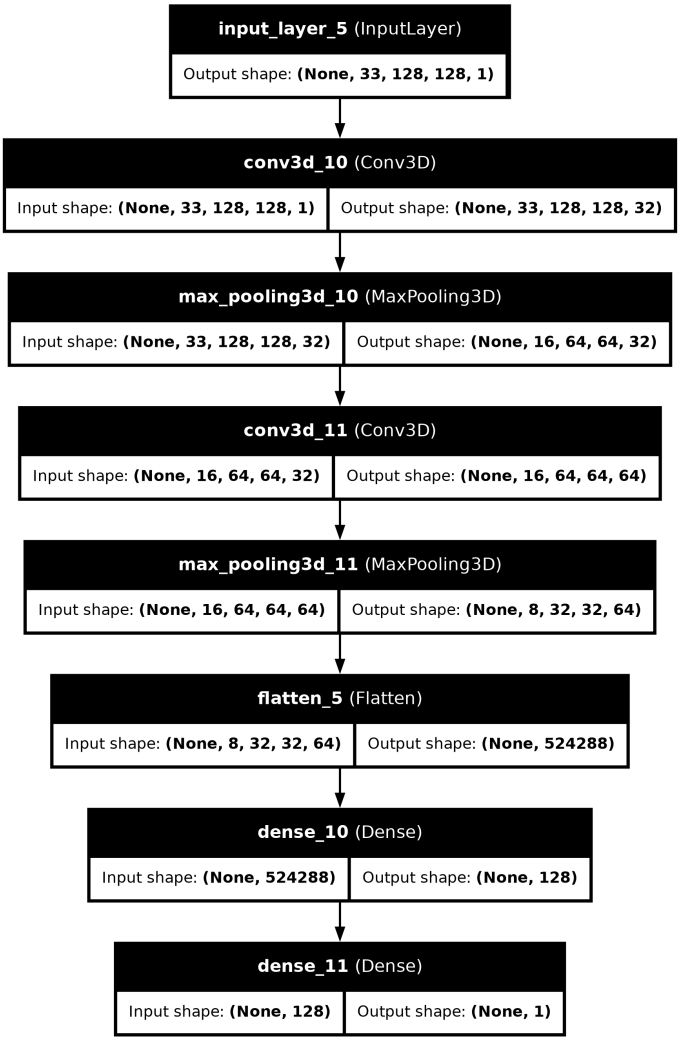


Figure B.1: First CNN Prototype with two Convolutional Block With MaxPooling and Flatten Layer with One Output Layer

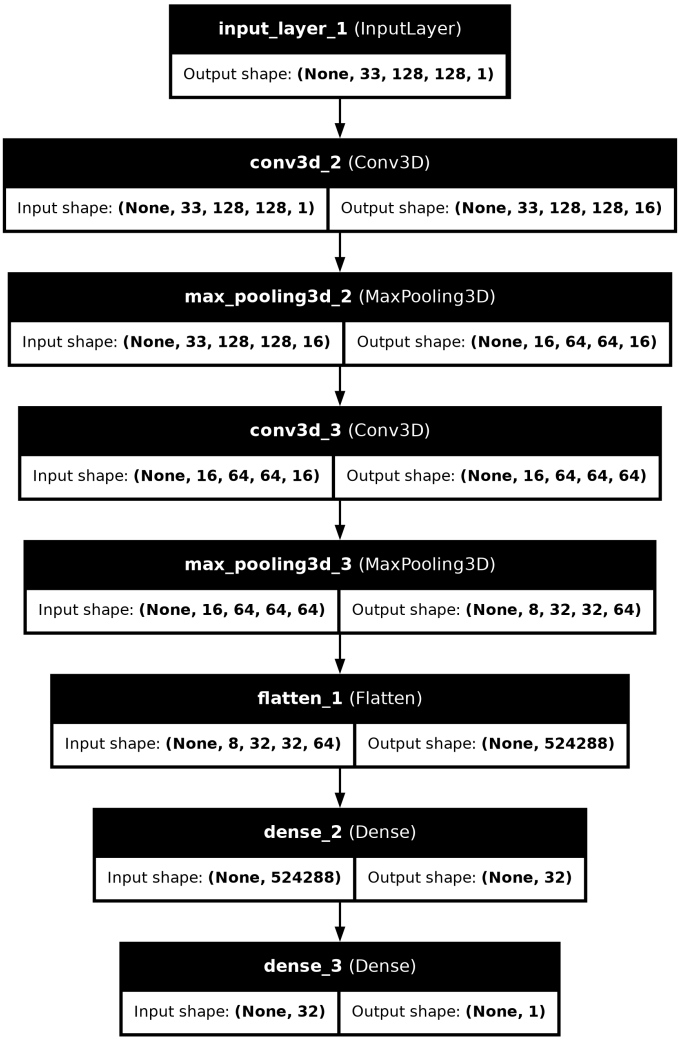
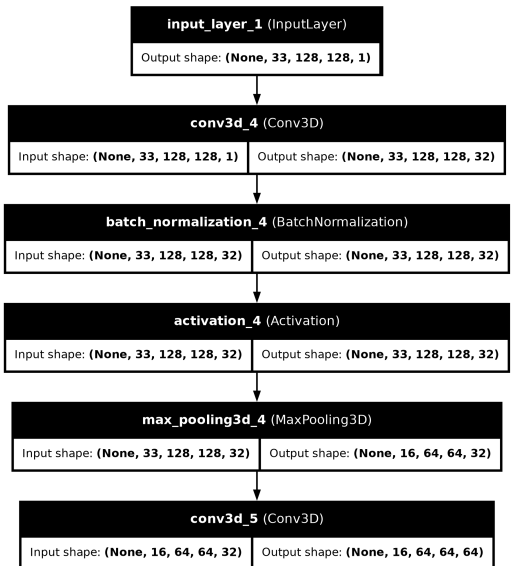
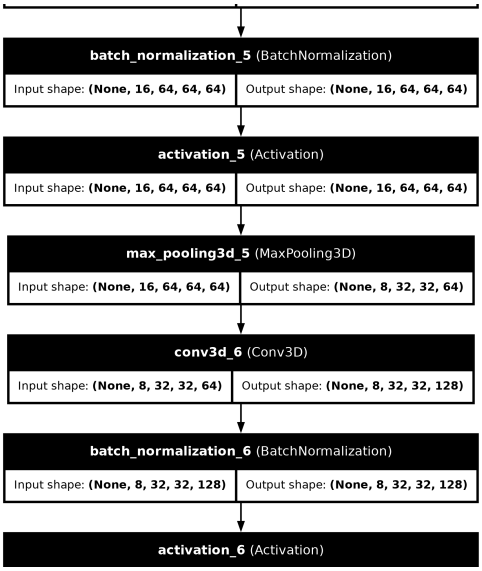


Figure B.2: First CNN Prototype with hyperparameters selected by Keras Tuner

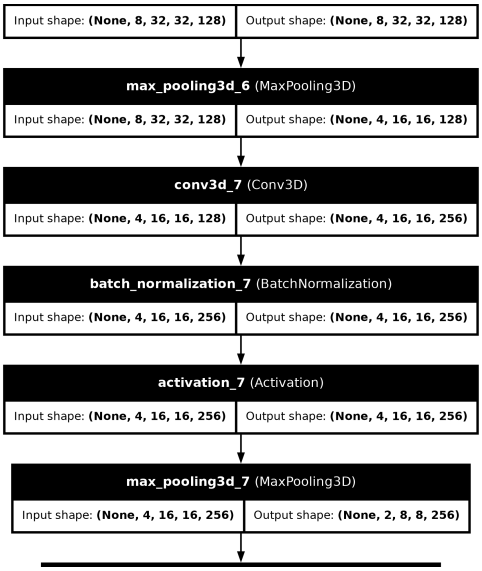
B.2 Model 2: CVSNet-Inspired



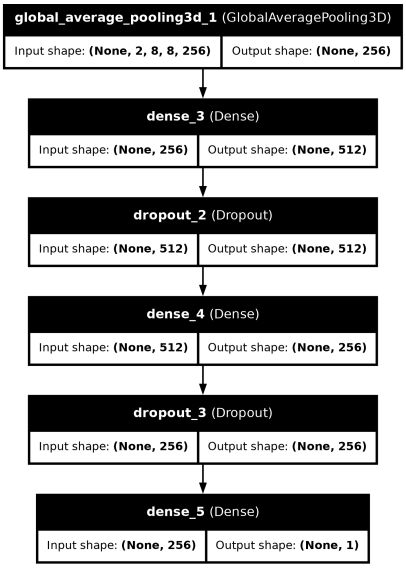
(a) Part 1



(b) Part 2

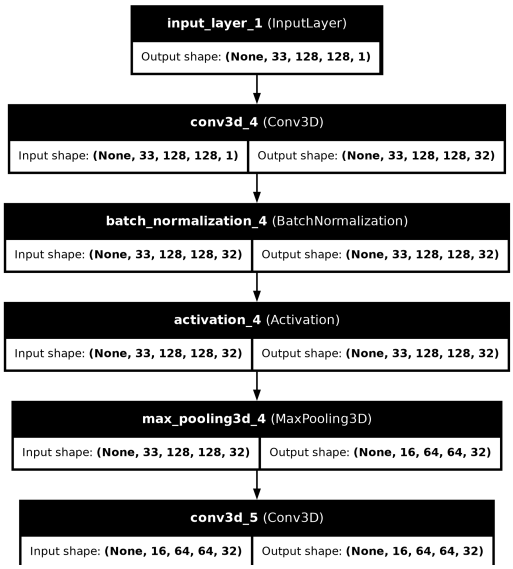


(c) Part 3

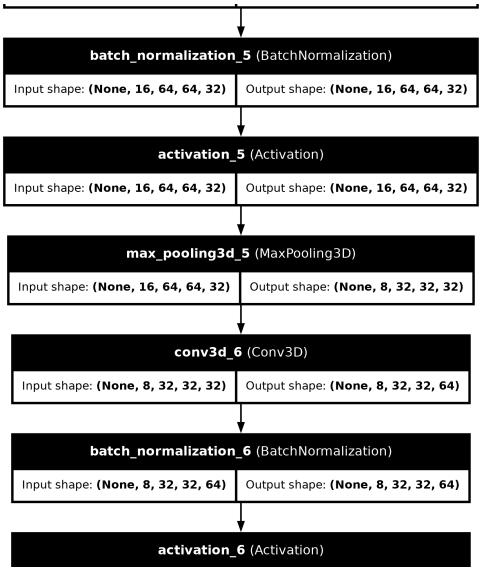


(d) Part 4

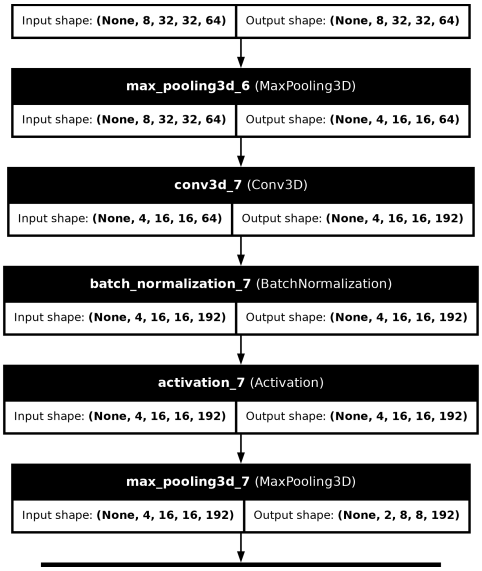
Figure B.3: Second Model inspired on CVSNet



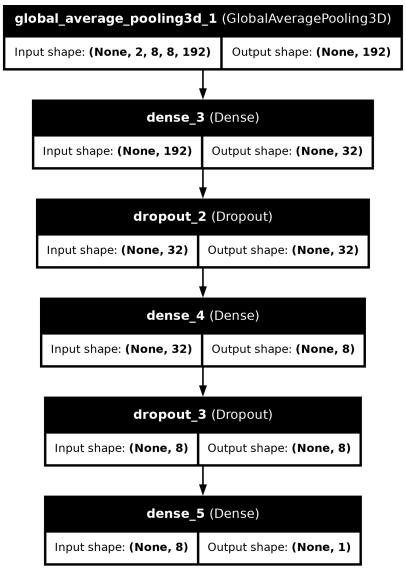
(a) Part 1



(b) Part 2



(c) Part 3



(d) Part 4

Figure B.4: Second Model inspired on CVSNet with hyperparameters selected by Keras Tuner

B.3 Model 3: Transfer Learning with ResNet50

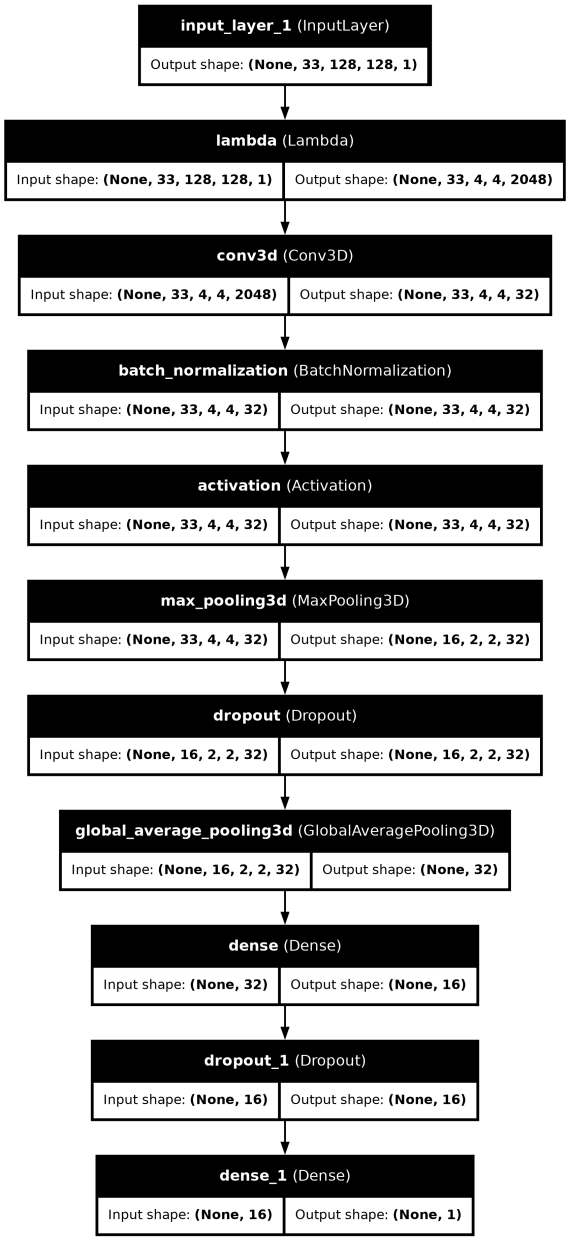


Figure B.5: CNN with Transfer Learning Model

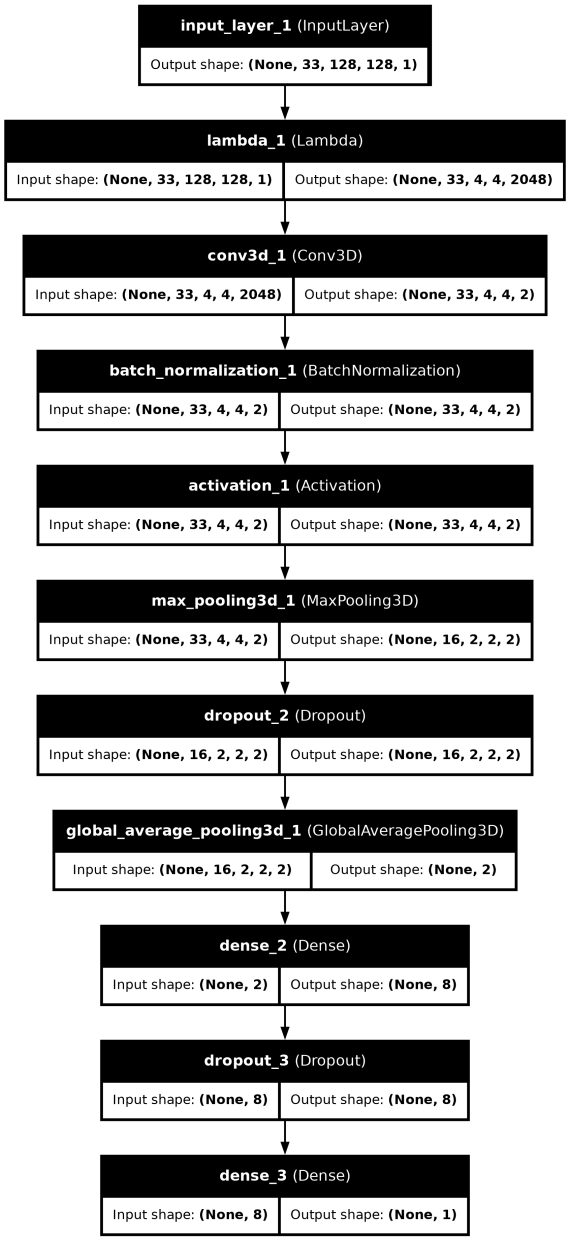


Figure B.6: CNN with Transfer Learning Model by Keras Tuner

Bibliography

Al-Louzi, O., Letchuman, V., Manukyan, S., Beck, E. S., Roy, S., Ohayon, J., Pham, D. L., Cortese, I., Sati, P., y Reich, D. S. (2022). Central vein sign profile of newly developing lesions in multiple sclerosis. *Neurology Neuroimmunology & Neuroinflammation*, 9(2):e1120.

David H Miller, Declan T Chard, O. C. (2012). Clinically isolated syndromes. *The Lancet Neurology*, 11.

Dworkin, J. D., Sati, P., Solomon, A., Pham, D., Watts, R., Martin, M., Ontaneda, D., Schindler, M., Reich, D., y Shinohara, R. (2018). Automated integration of multimodal mri for the probabilistic detection of the central vein sign in white matter lesions. *American Journal of Neuroradiology*, 39:1806 – 1813.

Egan, P. G. (2013). Investigation of tremor in multiple sclerosis (ms) via magnetic resonance imaging (mri). Eye and Ear Hospital Human Research Ethics Committee, ethics-id: 2008.022, Number of Subjects: 14, Number of Studies: 12, Number of Datasets: 126.

Filippi, M., Preziosa, P., Banwell, B. L., Barkhof, F., Ciccarelli, O., De Stefano, N., Geurts, J. J., Paul, F., Reich, D. S., Toosy, A. T., Traboulsee, A., Wattjes, M. P., Yousry, T. A., Gass, A., Lubetzki, C., Weinshenker, B. G., y Rocca, M. A. (2019). Assessment of lesions on magnetic resonance imaging in multiple sclerosis: practical guidelines. *Brain*, 142(7):1858–1875.

La Rosa, F., Wynen, M., Al-Louzi, O., Beck, E. S., Huelnhagen, T., Maggi, P., Thiran, J.-P., Kober, T., Shinohara, R. T., Sati, P., Reich, D. S., Granziera, C., Absinta, M., y Bach Cuadra, M. (2022). Cortical lesions, central vein sign, and paramagnetic rim lesions in multiple sclerosis: Emerging machine learning techniques and future avenues. *NeuroImage: Clinical*, 36:103205.

Loizou, C., Kyriacou, E., Seimenis, I., Pantziaris, M., Petroudi, S., Karaolis, M., y Pattichis, C. (2013a). Brain white matter lesion classification in multiple sclerosis subjects for the prognosis of future disability. *Intelligent Decision Technologies Journal (IDT)*, 7:3–10.

Loizou, C., Murray, V., Pattichis, M., Seimenis, I., Pantziaris, M., y Pattichis, C. (2011). Multi-scale amplitude modulation-frequency modulation (am-fm) texture analysis of multiple sclerosis in brain mri images. *IEEE Transactions on Information Technology in Biomedicine*, 15(1):119–129.

Loizou, C., Pantziaris, M., Pattichis, C., y Seimenis, I. (2013b). Brain mri image normalization in texture analysis of multiple sclerosis. *Journal of Biomedical Graphics and Computing*, 3(1):20–34.

Loizou, C., Petroudi, S., Seimenis, I., Pantziaris, M., y Pattichis, C. (In press). Quantitative texture analysis of brain white matter lesions derived from t2-weighted mr images in ms patients with clinically isolated syndrome. *Journal of Neuroradiology*. Accepted.

Loizou, C. P. (2018). Mri lesion segmentation in multiple sclerosis database. *Medinfo*.

M. Muslim, A. (2022). Brain mri dataset of multiple sclerosis with consensus manual lesion segmentation and patient meta information.

Maggi, P., Fartaria, M. J., Jorge, J., Rosa, F. L., Absinta, M., Sati, P., Meuli, R., Pasquier, R. D., Reich, D., Cuadra, M., Granziera, C., Richiardi, J., y Kober, T. (2020). Cvsnet: A machine learning approach for automated central vein sign assessment in multiple sclerosis. *NMR in Biomedicine*, 33.

Malinin, A., Athanasopoulos, A., Barakovic, M., Bach Cuadra, M., Gales, M., Granziera, C., Graziani, M., Kartashev, N., Kyriakopoulos, K., Lu, P.-J., Molchanova, N., Nikitakis, A., Raina, V., La Rosa, F., Sivena, E., Tsarsitalidis, V., Tsompopoulou, E., y Volf, E. (2022). Shifts multiple sclerosis lesion segmentation dataset part 2 (1.0). Data set.

National Institute of Biomedical Imaging and Bioengineering (NIBIB) (2023). Magnetic resonance imaging (mri). https://www.nibib.nih.gov/science-education/science-topics/magnetic-resonance-imaging-mri. Accessed: 2024-09-02.

Ontaneda, D., Sati, P., Raza, P., Kilbane, M., Gombos, E., Alvarez, E., Azevedo, C., Calabresi, P., Cohen, J., Freeman, L., Henry, R., Longbrake, E., Mitra, N., Illenberger, N., Schindler, M., Moreno-Dominguez, D., Ramos, M., Mowry, E., Oh, J., Rodrigues, P., Chahin, S., Kaisey, M., Waubant, E., Cutter, G., Shinohara, R.,

Reich, D., Solomon, A., y Sicotte, N. (2021). Central vein sign: A diagnostic biomarker in multiple sclerosis (cavs-ms) study protocol for a prospective multicenter trial. *NeuroImage: Clinical*, 32:102834.

Ortiz, G., Alvarado, L., Pacheco-Moises, F., Mireles-Ramírez, M., González, E., Sánchez-López, A., Sánchez-Romero, L., Santoscoy, J., Velázquez-Brizuela, I., y Sánchez González, V. J. (2016). Cross-talk between glial cells and neurons: Relationship in multiple sclerosis. *Clinical Case Reports and Reviews*, 2.

Sati, P., Oh, J., Constable, R. T., Evangelou, N., Guttmann, C. R., Henry, R. G., Klawiter, E. C., Mainero, C., Massacesi, L., Prineas, J. W., Rocca, M. A., Sahraian, M. A., Sormani, M. P., Vukusic, S., Yousry, T. A., Bakshi, R., Barkhof, F., Bove, R. M., Ciccarelli, O., Filippi, M., Inglese, M., Montalban, X., Pelletier, D., Reich, D. S., Samson, R. S., Silver, N. C., Traboulsee, A., Wattjes, M. P., y Geurts, J. J. (2016). The central vein sign and its clinical evaluation for the diagnosis of multiple sclerosis: a consensus statement from the north american imaging in multiple sclerosis cooperative. *Nature Reviews Neurology*, 12:714–722.

Sinnecker, T., Clarke, M., Meier, D., Enzinger, C., Calabrese, M., de Stefano, N., Pitiot, A., Giorgio, A., Schoonheim, M., Paul, F., Pawlak, M., Schmidt, R., Kappos, L., Montalban, X., Rovira, A., Evangelou, N., y Wuerfel, J. (2019). Evaluation of the central vein sign as a diagnostic imaging biomarker in multiple sclerosis. *JAMA neurology*.

Zhang, H. y Qie, Y. (2023). Applying deep learning to medical imaging: A review. *Applied Sciences*.

Ömerhoca, S., Akkaş, S. Y., y İçen, N. K. (2018). Multiple sclerosis: Diagnosis and differential diagnosis. *Noro Psikiyatr Ars*, 55:S1–S9.